KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

Project-based Practical Report on

"Sandboxed: A Linux-Based Isolated Program Execution and Resource Monitoring System"

Submitted in partial fulfilment of the requirements for "Operating Systems and Linux Core Programming" for the 1st Semester

MASTER OF SCIENCE IN CYBER SECURITY

Submitted by

Chris Mathew Aje

26CYBR10

Under the guidance of

Dr. Nithya B

November 2026

ABSTRACT

Running unverified Python scripts on a shared application server exposes user files, system credentials, outbound network sockets, host memory, and CPU cores to accidental exhaustion or deliberate misuse. Sandboxed confronts this risk without heavy virtual machines or privileged container daemons. Instead, it constructs an unprivileged Linux execution cage out of bubblewrap (bwrap) namespaces, control groups version 2 (cgroups v2), declarative seccomp profiles, and a bounded POSIX process supervisor.

Every containment knob sits in plain sight. Through an ExecutionPolicy schema, operators fix wall-clock deadlines, RAM ceilings, swap bans, Completely Fair Scheduler (CFS) CPU percentages, process count limits, stream capture caps, filesystem views, and network policies. Three codebases work together inside the repository: a React 18 browser console (apps/web), a FastAPI and SQLAlchemy service persisted to SQLite (apps/api), and a standalone execution package (sandbox_engine). Submitted jobs wait inside an async FIFO queue capped at 50 entries and run one at a time so CPU quotas do not distort neighbouring tasks.

Safety starts with refusal rather than silent fallback. Before spawning any target script, LinuxSandboxRunner calls detect_capabilities() to check the kernel release, bwrap binary, user namespace sysctls, unified cgroup2 mounts, delegated write permissions inside sandboxed.slice, and seccomp support. When the control plane runs on Windows or misses a delegated Linux prerequisite, the runner throws SandboxCapabilityError and marks the job FAILED with PREPARATION_ERROR instead of executing bare Python directly on the host. When Linux checks pass, ProcessSupervisor establishes an independent session via os.setsid(), captures bounded non-blocking stdout and stderr pipes, enforces monotonic wall-clock limits across the full process group, and reads cpu.stat, memory.peak, memory.events, and pids.events before removing the per-job slice and temporary directory.

Verification follows an honest two-stage split. On the Windows 11 workstation (Python 3.13.14), running pytest collects 53 cases: 43 unit, policy, SQLite persistence, REST endpoint, bubblewrap command, cgroups arithmetic, seccomp profile, and state-machine tests pass in 8.06 seconds, while the 10 Linux kernel integration tests remain explicitly skipped via sys.platform checks. Live screenshots from the React UI and Swagger documentation verify end-to-end API operation, and idempotent shell scripts prepare Ubuntu 22.04 or 24.04 LTS hosts for full kernel verification.

CONTENTS

SECTION / CHAPTER	PAGE NO.
1. Introduction	**1**
1.1 Background and Motivation	1
1.2 Problem Formulation	1
1.3 Kernel Primitives for Process Containment	2
1.4 Literature Review and Comparative Isolation Approaches	2
1.5 Motivation for Unified Policy Experimentation	3
1.6 Organisation of the Report	4
2. Objectives of the Project	**5**
2.1 Core Isolation and Least-Privilege Objectives	5
2.2 Resource Governance and Watchdog Objectives	5
2.3 Observability and Pedagogical Objectives	5
2.4 Threat Model and Security Boundary Matrix	6
3. Scope of the Project	**8**
3.1 Supported Runtimes and Functional Scope	8
3.2 Host Environment Assumptions and Kernel Prerequisites	8
3.3 Dual-Platform Semantics (Windows Control Plane vs Linux Execution)	8
3.4 Operational Safety and Ethical Boundaries	9
3.5 Repository Structure and Architectural Separation	9
4. Software and System Specifications	**11**
4.1 Target Operating Environment and Host Tooling	11
4.2 Backend Framework and Core Engine Separation	11
4.3 Policy Validation Envelope and Boundary Constraints	11
4.4 Runtime Configuration and Persistence Environment	12
4.5 Deployment Provisioning and Application Hardening	13
5. System Architecture and Workflow	**14**
5.1 End-to-End Job Submission and Policy Resolution	14
5.2 Host Capability Probing and Fail-Secure Gating	15
5.3 Per-Execution Cgroup Allocation and Bubblewrap Construction	15
5.4 Supervised Process Execution and Watchdog Escalation	16
5.5 Telemetry Harvesting, Cleanup, and Lifecycle Finalisation	16
5.6 Relational Persistence and Immutable Policy Snapshots	17
5.7 Asynchronous Callback Synchronisation and Startup Recovery	18
6. Modules of the Project	**20**

SECTION / CHAPTER	PAGE NO.
6.1 Policy Domain Models and Default Profiles	20
6.2 Built-in Academic Benchmark Workloads	20
6.3 Host Capability Detector	21
6.4 Bubblewrap Command Builder	22
6.5 Cgroups v2 Resource Manager	22
6.6 Seccomp Syscall Profile Definitions	22
6.7 Execution Runner and Supervision Engine	23
6.8 Asynchronous Queue Manager and State Machine	23
6.9 Cgroups v2 Telemetry Collector	24
6.10 FastAPI Data Repositories, Services, and REST Endpoints	24
6.11 React Presentation Workspace	25
7. Implementation and Source Code	**27**
7.1 Host Capability Probing and Honest Non-Linux Reporting	27
7.2 Policy Constraints and Validation Rules	27
7.3 Bubblewrap Namespace and Mount Assembly	28
7.4 Cgroups v2 Quota Configuration and Teardown	28
7.5 Seccomp Syscall Denylist Profiles	29
7.6 Runner Orchestration and Guaranteed Cleanup	29
7.7 Non-Blocking Supervision and Watchdog Escalation	30
7.8 Kernel Counter Parsing and Telemetry Conversion	31
7.9 Async Worker Dispatch and State Mapping	31
7.10 Sequential Policy Lab Execution and Persistence Bridge	32
7.11 Asynchronous Callback Synchronisation and Lifespan Recovery	32
7.12 Ubuntu Deployment Provisioning and Verification Procedure	33
8. System Verification, Features, and Interface Walkthrough	**34**
8.1 Overview Dashboard and Capabilities Status Banner	34
8.2 Interactive Execution Playground and Policy Catalogue	34
8.3 Execution Audit Log and Stream Inspector	36
8.4 Policy Lab Comparative Experiments and Host Diagnostics	36
8.5 Automated Test Verification Summary (Windows Phase)	39
8.6 Ubuntu Linux Kernel Benchmark Protocol and Troubleshooting Guide	40
9. Future Enhancements	**42**
9.1 Hardware-Virtualised Micro-VM Runners (Firecracker and gVisor)	42
9.2 Runtime Seccomp BPF Compilation and Landlock Integration	42
9.3 Multi-Language Compiler Toolchains	42

SECTION / CHAPTER	PAGE NO.
9.4 Real-Time WebSocket Stream Tail and Multi-Node Broker Scheduling	43
10. Conclusion	**44**
10.1 Engineering Synthesis and Operating Systems Insights	44
10.2 Known Constraints and Architectural Tradeoffs	44
10.3 Final Summary of Outcomes	45
References	**46**

LIST OF FIGURES

FIGURE	PAGE NO.
Figure 2.1: Defense-in-Depth Attack Vector to Kernel Containment Mapping	6
Figure 5.1: Sandboxed Multi-Layer System Architecture and Kernel Containment	14
Figure 5.2: ExecutionManager Lifecycle State Machine and Signal Escalation Flow	16
Figure 5.3: SQLite Relational Persistence Schema (apps/api/app/models/entities.py)	18
Figure 8.1: System and Execution Overview Dashboard	34
Figure 8.2: Interactive Execution Playground	35
Figure 8.3: Security and Resource Policy Profiles	35
Figure 8.4: Execution Audit Log and Stream Inspector	36
Figure 8.5: Policy Lab Comparative Benchmarking Workspace	37
Figure 8.6: Live Host Capability Probe and Diagnostics	38
Figure 8.7: FastAPI OpenAPI 3.1 Interactive Documentation	38
Figure 8.8: Verified Pytest Execution Output (43 Passed, 10 Linux Checks Skipped)	39

LIST OF TABLES

TABLE	PAGE NO.
Table 1.1: Comparison of Process Isolation Strategies for Academic Workloads	3
Table 2.1: Threat Vector to Enforcement and Telemetry Mapping	7
Table 3.1: Repository Package Layout and Component Ownership	9
Table 4.1: Software and Kernel Component Specifications	11
Table 4.2: Server-Enforced Policy and Request Validation Boundaries	12
Table 4.3: Backend Environment Configuration Defaults (config.py)	12
Table 5.1: Execution Lifecycle Stages and Component Ownership	17
Table 5.2: SQLite Database Tables, Keys, and Index Design	18
Table 6.1: Default Security and Resource Policy Profiles (DEFAULT_POLICIES)	20
Table 6.2: Built-in Operating Systems Test Workloads (BUILTIN_WORKLOADS)	21
Table 6.3: Complete FastAPI REST Endpoint Catalogue	25
Table 6.4: React Frontend Views and Functional Responsibilities	25
Table 8.1: Backend API, Persistence, and Policy Test Inventory (18 Passed on Windows)	39
Table 8.2: Core Sandbox Engine Unit Test Inventory (25 Passed on Windows)	40
Table 8.3: Linux Kernel Integration Tests & Ubuntu Benchmark Protocol (10 Pending Linux Run)	41
Table 8.4: Ubuntu Host Verification Troubleshooting Matrix	41

1. Introduction

1.1 Background and Motivation

University coding labs, online judges, and incident response sandboxes share an awkward constraint: they must execute code whose behaviour is unknown ahead of time on hardware that still belongs to the host operator. Even ordinary student bugs carry real operational cost. A missing termination check keeps a CPU core pinned at 100 percent indefinitely, an unchecked byte buffer allocates RAM until host services page to disk, and a misconfigured recursion spawns worker trees faster than an instructor can kill them by hand.

Deliberate payloads raise the stakes much further. A guest script running as an ordinary subprocess can walk environment variables, probe SSH keys under ~/.ssh, inspect host databases sitting on local disk, or open raw TCP sockets against internal campus services. Any execution harness built for Operating Systems education therefore has two jobs at once. First, it must restrict visibility and hardware usage cleanly. Second, it must surface clear kernel telemetry so learners observe exactly why a process succeeded, throttled under scheduler pressure, hit an out-of-memory kill, or failed an access check.

Conventional tools often hide these internals behind opaque abstractions. When students submit code to a managed cloud runner or a turnkey Docker container, Linux namespaces, cgroups v2 control files, and signal propagation remain invisible behind YAML manifests or background daemons. Sandboxed closes that gap by treating kernel boundaries as interactive, logged experiment parameters attached to every job.

1.2 Problem Formulation

Launching unverified Python interpreters via bare subprocess calls exposes the underlying operating system to four structural failures that drove the design of this project:

- **Unrestricted host filesystem visibility:** Processes sharing the host mount table inherit the user's discretionary access across SSH credentials, shell histories, application SQLite databases, and read-only system trees.
- **Memory starvation and process floods:** Without hard RAM caps, disabled swap spillover, and task ceilings, aggressive allocations degrade every concurrent daemon on the machine and may trigger the global OOM killer on vital system processes.

- **Unchecked socket and DNS egress:** A script running inside the host network stack can initiate outbound TCP traffic, scan loopback daemons bound to localhost ports, or leak hostname metadata through DNS queries.
- **Opaque failure modes and pipe deadlocks:** Basic runners confuse OOM kills with syntax exits, fail to reap forked grandchildren when parents terminate early, or deadlock on full OS pipe buffers when guest code floods standard output.

1.3 Kernel Primitives for Process Containment

Mainline Linux kernels split containment across orthogonal subsystems that inspect and constrain running tasks without hardware hypervisors. Namespaces govern what a process tree sees. Unsharing user, mount, PID, IPC, UTS, cgroup, and network namespaces gives an unprivileged guest interpreter its own private PID tree starting at PID 1, a fresh loopback interface, a sterile host name, and a tailored root directory.

Where namespaces control visibility, control groups version 2 govern physical hardware consumption. Writing byte ceilings to memory.max, zero swap allowance to memory.swap.max, microsecond quota pairs to cpu.max, and child ceilings to pids.max delegates hard accounting directly to the Linux scheduler and virtual memory manager. Reading cpu.stat, memory.peak, memory.events, and pids.events turns kernel enforcement into exact post-run numbers rather than coarse process-level guesses.

Seccomp BPF filtering narrows the third attack surface: system calls crossing from user mode into kernel mode. Blocking tracing primitives (ptrace), filesystem mounting (mount), BPF loading (bpf), kernel module insertion, and optional socket calls closes common container-escape vectors even before kernel code runs.

1.4 Literature Review and Comparative Isolation Approaches

Selecting an execution mechanism requires balancing startup latency, operational weight, security guarantees, and observability. Full Type-1 or Type-2 hypervisors (such as KVM guests managed via QEMU) assign each task an independent guest kernel, yielding strong tenant boundaries at the price of gigabytes of image overhead and multi-second boot delays. For short interactive classroom tests that run in milliseconds, spawning a fresh virtual machine per submission wastes far more CPU booting systemd than running the student script.

At the opposite extreme, language-level sandboxes built inside CPython attempt to strip __builtins__ or filter AST nodes around eval(). Decades of documented escapes show why Python-level confinement fails: C extensions, object introspection attributes (__subclasses__, __mro__), and frame navigation repeatedly bypass interpreter restrictions. Legacy chroot jails help mask directory paths yet provide zero PID, IPC, network, or CPU isolation and traditionally demand elevated privileges.

Container daemons such as Docker Engine combine namespaces and cgroups automatically, yet introduce a root-owned long-running daemon, overlay storage drivers, and image registries that complicate local university labs. By contrast, unprivileged bubblewrap (bwrap) constructs identical kernel namespaces in a single zero-daemon execve invocation once unprivileged user namespaces are available. Micro-VM platforms such as Firecracker push isolation further for untrusted cloud SaaS tenants, whereas unprivileged bwrap paired with delegated cgroups v2 offers transparent inspection ideal for OS coursework.

Table 1.1: Comparison of Process Isolation Strategies for Academic Workloads

Isolation Mechanism	Privilege / Daemon Needs	Kernel & Resource Controls	Suitability for OS Lab Use
Python eval() Wrapper	Unprivileged user process	None (trivial C-extension / MRO bypasses)	Unsafe; zero OS telemetry
Legacy chroot + ulimit	Requires root / CAP_SYS_CHROOT	Filesystem prefix only; RLIMITs lack tree caps	Weak; no PID, net, or OOM accounting
Hardware VM (QEMU / KVM)	Hypervisor + guest disk images	Full hardware VM boundary; separate kernel	Heavy RAM footprint and slow startup
Docker / containerd	Background root/rootless daemon	Namespaces, cgroups, overlayfs, default seccomp	Masks raw cgroup/namespace internals
Firecracker Micro-VM	Host KVM (/dev/kvm) access	Minimal VMM + guest kernel per workload	Strongest isolation; complex classroom setup
Sandboxed (bwrap + cgroups v2)	Zero daemon; delegated user slice	Explicit namespaces, cgroup2 counters, seccomp	Direct kernel visibility and low overhead

1.5 Motivation for Unified Policy Experimentation

Building isolation flags in a shell script is rarely enough for learning. When a student runs a prime-search loop at 100 percent CPU versus 20 percent CFS bandwidth quota, or allocates 50 MiB of RAM inside a 128 MiB cage versus a 32 MiB cage, reading raw text counters in /sys/fs/cgroup requires tedious manual shell work before the directory disappears. Pairing Sandboxed with a browser workspace, a persistent SQLite audit history, and an automated Policy Lab turns those OS mechanisms into repeatable comparative experiments.

1.6 Organisation of the Report

The remaining chapters trace the architecture, implementation, and validation of Sandboxed in detail. Chapter 2 states the isolation, resource governance, pedagogical objectives, and threat model. Chapter 3 fixes technical scope and dual-platform assumptions. Chapter 4 inventories software versions, Pydantic validation bounds, and configuration settings. Chapters 5 and 6 examine end-to-end runtime workflow, the SQLite relational schema, and module internals. Chapter 7 documents annotated source code listings and Ubuntu setup scripts. Chapter 8 presents live web interface captures, the 53-case pytest inventory, and the Ubuntu kernel benchmark procedure. Chapters 9 and 10 conclude with planned extensions and honest engineering limits.

2. Objectives of the Project

2.1 Core Isolation and Least-Privilege Objectives

The foremost engineering goal is isolating untrusted Python 3 code under unprivileged Linux execution. Target workloads must run without giving root access to the web API daemon and without installing a privileged container runtime. Using bubblewrap, each invocation enters private user, mount, PID, IPC, UTS, and cgroup namespaces. Host credential trees (/root, /home, /etc/shadow) are withheld completely so scripts see only read-only OS libraries and a blank ephemeral working folder mounted at /workspace.

Fail-secure gating stands on equal footing with mount containment. Many student prototypes silently drop to bare subprocess.run() calls if sandboxing tools are unavailable on a laptop. Sandboxed explicitly rejects that anti-pattern. Before launching any user payload, detect_capabilities() verifies the host OS and required kernel subsystems, raising SandboxCapabilityError whenever a prerequisite is missing.

2.2 Resource Governance and Watchdog Objectives

A complete sandbox must govern time, memory, scheduler bandwidth, task count, and log volume together. Every job inherits bounds validated by Pydantic on the server: wall clocks capped between 0.1 and 60.0 seconds, RAM locked between 16 and 2048 MiB with swap disabled via memory.swap.max = 0, CFS CPU quotas scaled from 10 to 100 percent, PID ceilings between 1 and 128 tasks, and byte capture caps up to 1 MiB.

Deterministic teardown is just as vital as quota allocation. If an untrusted parent script spawns worker subprocesses before locking up, killing only the top PID leaks orphaned children across the host. Running the supervisor inside a fresh session (os.setsid()), monitoring monotonic elapsed deadlines every 20 ms, and sending SIGTERM followed by SIGKILL to the full process group (-pgid) plus invoking cgroup.kill guarantees that every descendant stops cleanly before temporary storage is removed.

2.3 Observability and Pedagogical Objectives

Sandboxed treats execution metrics as first-class experimental data. Rather than returning a bare exit code, the collector splits user-space CPU seconds from kernel-space system CPU seconds, converts memory.peak into megabytes, captures oom_kill flags from memory.events, and computes SHA-256 digests of source code alongside immutable policy

JSON snapshots inside SQLite. Students can therefore replay, inspect, and compare identical scripts across contrasting policies using interactive charts inside Policy Lab.

2.4 Threat Model and Security Boundary Matrix

Clear threat modelling avoids exaggerated security claims. In Sandboxed, the web operator and Linux host kernel reside inside the trusted computing base (TCB). Submitted Python scripts and uploaded .py payloads are treated as fully untrusted user-space code capable of disk traversal, heap exhaustion, infinite busy loops, recursive process spawning, pipe buffer flooding, and outbound socket probes.

As mapped in Figure 2.1, six primary threat classes terminate cleanly at specific namespace, cgroup, or supervisor boundaries before harming host integrity. Zero-day local privilege escalation bugs inside the shared host Linux kernel itself or speculative hardware side channels remain outside user-space container scope and motivate the micro-VM roadmap documented in Chapter 9.

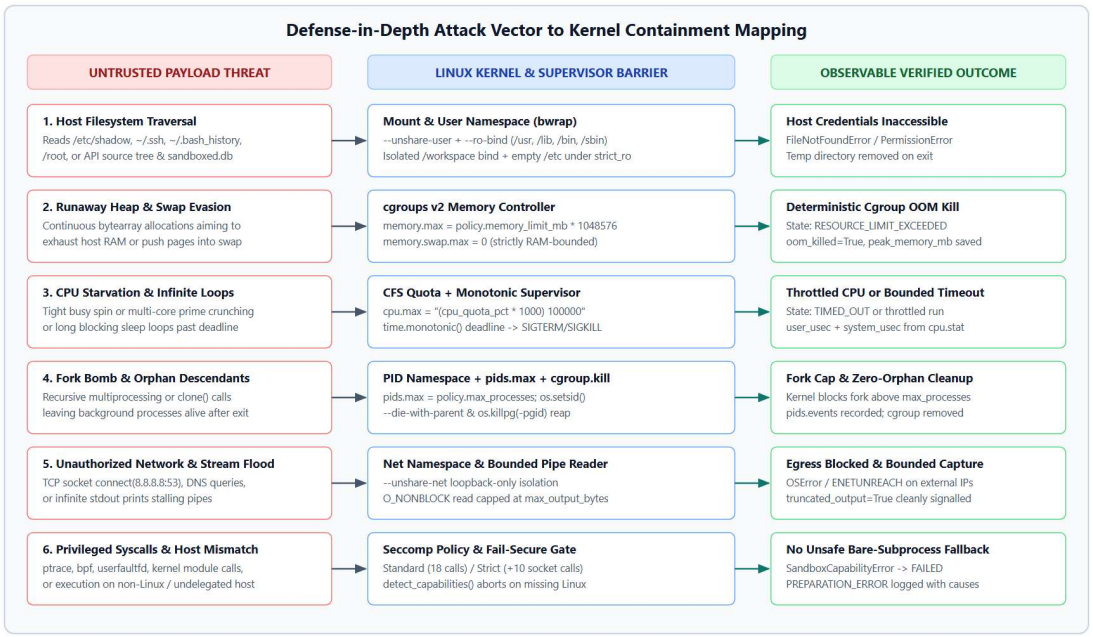


Figure 2.1: Defense-in-Depth Attack Vector to Kernel Containment Mapping

Description: Six-row security mapping tracing untrusted Python payload threats (host traversal, heap and swap abuse, CPU spin, fork bombs, network egress, and privileged syscalls) to specific bwrap, cgroups v2, seccomp, and supervisor controls and their recorded execution states.

Table 2.1: Threat Vector to Enforcement and Telemetry Mapping

Threat Vector	Sandbox Attack Example	Primary Enforcing Primitive	Logged State & Metric
Host File Exposure	Opening /etc/shadow, ~/.ssh, or sandboxed.db	bwrap mount namespace + RO system binds	PermissionError / FileNotFoundError; clean teardown
Swap-Assisted RAM Bomb	Allocating 50+ MiB bytearray chunks inside 32 MiB	cgroups v2 memory.max & memory.swap.max=0	RESOURCE_LIMIT_EXCEEDED; oom_killed=True
CFS CPU Starvation	Tight prime search pinning host CPU cores	cgroups v2 cpu.max + supervisor deadline	TIMED_OUT or throttled cpu_usage_total_sec
Fork Flood / Orphan Leak	Spawning 50 child workers via multiprocessing	PID namespace + pids.max + -pgid / cgroup.kill	Kernel halts fork above cap; zero orphan tasks
Unauthorized Net Egress	TCP socket.connect(('8.8.8.8', 53))	bwrap --unshare-net + STRICT_RO /etc tmpfs	ENETUNREACH / OSError in script stdout/stderr
Sys Call & Host Mismatch	ptrace/bpf calls or running on non-Linux host	Declarative seccomp defs + capability probe	PREPARATION_ERROR on missing host Linux layer

3. Scope of the Project

3.1 Supported Runtimes and Functional Scope

The current release targets isolated execution of Python 3 source programs passed directly through the Playground code editor or uploaded as .py files up to 100,000 characters long. Policies configure three filesystem isolation modes (isolated_ro_root, minimal, and strict_ro), three syscall profile enums (standard, strict, and disabled), and a boolean network toggle that enforces --unshare-net by default.

Six built-in policies ship pre-seeded in SQLite (standard, strict, cpu_constrained, memory_constrained, network_blocked, and permissive), paired with eight curated academic workloads covering arithmetic baselines, prime-number CPU benchmarking, 50 MiB chunked RAM growth, 20-second deadline timeouts, file escape checks, external TCP socket probes, high-volume stream flooding, and multi-process fork limits. Users may author custom policies through the UI and benchmark up to six policies per Policy Lab experiment.

3.2 Host Environment Assumptions and Kernel Prerequisites

Active kernel containment assumes Ubuntu 22.04 LTS or Ubuntu 24.04 LTS (x86_64) running Linux kernel 5.15 or newer with a unified cgroups v2 hierarchy mounted at /sys/fs/cgroup. Both /sys/fs/cgroup/cgroup.controllers and delegated subtree permissions under sandboxed.slice (or user.slice) must expose cpu, memory, and pids controllers. Unprivileged user namespaces must be usable by the non-root operator account running Uvicorn.

On Ubuntu 24.04 hosts carrying stricter AppArmor unprivileged user namespace profiles (controlled via kernel.apparmor_restrict_unprivileged_userns), administrators must ensure bubblewrap retains permission to enter unprivileged user namespaces through packaged AppArmor rules or explicit setup provisioning. Where those prerequisites fail, detect_capabilities() appends explicit remediation guidance to the UI banner.

3.3 Dual-Platform Semantics (Windows Control Plane vs Linux Execution)

Software engineering for Linux kernel systems routinely proceeds on developer laptops before deployment to target Linux hosts or VMware VMs. Sandboxed mirrors this workflow cleanly. Its FastAPI control server, SQLAlchemy repository layer, Pydantic policy validator, React frontend, and 43 platform-independent tests run natively on Windows 11 with zero mock illusions.

Crucially, Windows hosts do not possess Linux namespaces, cgroups v2 pseudo-files, or /usr/bin/bwrap. Rather than stubbing fake execution success on Windows, detect_capabilities() returns is_linux=False and ready=False, surfacing a persistent Host: Non-Linux indicator in the navigation rail and refusing live program execution cleanly. Live containment metrics are gathered only when deployed on Ubuntu Linux.

3.4 Operational Safety and Ethical Boundaries

Sandboxed is scoped strictly for academic Operating Systems coursework, controlled automated grading, and reproducible resource-limit experiments on dedicated lab hosts or virtual machines. Queue concurrency stays fixed at MAX_CONCURRENCY=1 over a 50-slot bounded asyncio.Queue so sibling jobs never contend for host CFS cycles or distort peak memory readings. Because containers share one host kernel, high-risk zero-day malware detonation or multi-tenant public internet hosting sits outside the security boundary of this version.

Every transient artifact produced during a run follows deterministic cleanup. Both the per-execution folder /tmp/sandboxed_storage/exec_<execution_id> and its paired control group slice exec_<execution_id> unlink immediately after ProcessSupervisor collects kernel metrics, preventing stale files or zombie PID namespaces from piling up across long classroom lab sessions.

3.5 Repository Structure and Architectural Separation

Code organization mirrors the runtime isolation boundary so that low-level OS probes, HTTP handlers, and browser components stay independently testable. Nothing in sandbox_engine imports FastAPI or SQLAlchemy, whereas nothing inside apps/web calls Linux binaries directly. Table 3.1 summarizes directory ownership across the tree.

Table 3.1: Repository Package Layout and Component Ownership

Directory Path	Layer Role	Key Modules Contained
sandbox_engine/linux/	Kernel Primitives	capabilities.py, bwrap_builder.py, cgroup_v2.py, seccomp_filter.py, supervisor.py, runner.py
sandbox_engine/manager/	Queue & State Machine	execution_manager.py managing FIFO queue, worker loop, and cancellation flags
sandbox_engine/monitoring/	Telemetry Parser	collector.py translating cpu.stat, memory.peak, memory.events, and pids.events
sandbox_engine/policies/	Domain Models	models.py defining ExecutionPolicy constraints, DEFAULT_POLICIES, and BUILTIN_WORKLOADS

Directory Path	Layer Role	Key Modules Contained
apps/api/app/	REST API & ORM	main.py, deps.py, config.py, entities.py, repositories/, services/, api/routers/
apps/web/src/	Browser Interface	App.jsx, layouts/AppLayout.jsx, 7 routed React pages, Tailwind & CodeMirror UI
scripts/	Ops & Test Automation	setup_ubuntu.sh, verify_linux_sandbox.sh, run_dev.sh, run_tests.sh

4. Software and System Specifications

4.1 Target Operating Environment and Host Tooling

Control-plane development and automated unit testing were carried out on Windows 11 (Build 10.0.26200) using Python 3.13.14, pytest 9.2.0, Node.js, and Vite 5.4.14. Runtime Linux kernel enforcement targets Ubuntu 22.04 or 24.04 LTS on kernel 5.15+ with bubblewrap (/usr/bin/bwrap) and libseccomp2 installed via apt.

Unified cgroups v2 is non-negotiable. Legacy cgroups v1 split controllers across separate mount trees and lacked atomic cgroup.kill signalling, swapless memory.swap.max interfaces, and unified memory.peak watermarks. Every metric path in Sandboxed reads unified cgroup2 hierarchy paths exclusively.

4.2 Backend Framework and Core Engine Separation

The backend layer under apps/api uses FastAPI (>= 0.110.0), Uvicorn (>= 0.28.0), Pydantic (>= 2.6.0), and SQLAlchemy (>= 2.0.0) backed by SQLite (sandboxed.db). Keeping HTTP routers, ORM persistence, and async lifespan wiring separate from sandbox_engine lets engineers validate capability detection, bubblewrap CLI assembly, cgroup arithmetic, and supervisor state transitions directly inside unit tests with zero HTTP server.

Table 4.1: Software and Kernel Component Specifications

Layer / Component	Technology / Version	Primary Role in Sandboxed
Target OS & Kernel	Ubuntu 22.04 / 24.04 LTS (Linux 5.15+)	Provides unified cgroups v2, user namespaces, and seccomp
Container Runtime	bubblewrap (bwrap)	Constructs unprivileged mount, PID, IPC, UTS, cgroup, and net namespaces
Resource Governance	cgroups v2 (cpu, memory, pids)	Enforces memory.max, memory.swap.max=0, cpu.max, and pids.max
Backend API	FastAPI, Uvicorn, Pydantic v2	Validates policy bounds, manages lifecycle, exposes /api/* REST endpoints
Persistence Layer	SQLAlchemy 2.0 + SQLite (sandboxed.db)	Stores policies, SHA-256 code hashes, telemetry, experiments, and audit events
Frontend UI	React 18, Vite, Tailwind, CodeMirror, Recharts	Provides Overview, Playground, Executions, Policy Lab, Policies, and System Info
Verification	pytest 9.2, pytest-asyncio, httpx	Runs 43 Windows unit/API checks and 10 Linux kernel integration tests

4.3 Policy Validation Envelope and Boundary Constraints

Untrusted clients cannot pass arbitrary resource requests down to the OS kernel. Both the low-level ExecutionPolicy model and API request schemas (PolicyCreateRequest,

PolicyUpdateRequest, ExecutionCreateRequest, and ExperimentCreateRequest) reject out-of-range numbers with HTTP 400 or HTTP 422 before persistence or queueing occurs. Table 4.2 lists every checked schema boundary observed in code.

Table 4.2: Server-Enforced Policy and Request Validation Boundaries

Schema Field / Parameter	Default	Allowed Envelope	Operational & Security Rationale
PolicyCreateRequest.id	Required	2..64 chars, ^[a-zA-Z0-9_\-]+$	Prevents path traversal or shell characters in policy IDs
wall_time_limit_sec	5.0 s	0.1 s .. 60.0 s	Stops indefinite sleeps while allowing multi-second builds
memory_limit_mb	128 MiB	16 MiB .. 2048 MiB	Avoids instant Python init failure and caps host RAM risk
cpu_quota_pct	50%	10% .. 100%	Maps linearly to CFS microseconds inside a 100,000 us window
max_processes	32	1 .. 128 PIDs	Contains fork floods cleanly under cgroups v2 pids.max
max_output_bytes	65,536 B	1,024 B .. 1,048,576 B	Protects supervisor heap and SQLite rows from stdout floods
temp_storage_mb	64 MiB	8 MiB .. 512 MiB	Declares per-job scratch budget in /workspace policy metadata
ExecutionCreateRequest.source_code	Required	1 .. 100,000 characters	Rejects empty scripts and multi-megabyte JSON upload DoS
ExperimentCreateRequest.policy_ids	Required	1 .. 6 policy IDs	Caps sequential Policy Lab queues to reasonable run windows

4.4 Runtime Configuration and Persistence Environment

Backend configuration defaults live in apps/api/app/core/config.py inside a Pydantic Settings class, allowing environment variable overrides without editing application logic. SQLite runs with check_same_thread=False in apps/api/app/models/db.py so FastAPI request handlers and background worker callbacks can synchronize job states smoothly across threads. Table 4.3 documents all active settings.

Table 4.3: Backend Environment Configuration Defaults (config.py)

Setting Key	Default Value	System Component Governed
API_HOST / API_PORT	127.0.0.1 : 8000	Uvicorn local loopback bind address preventing unintended LAN exposure
CORS_ORIGINS	localhost:5173, 127.0.0.1:5173	Permits local Vite dev server cross-origin REST calls
DATABASE_URL	sqlite:///./sandboxed.db	SQLAlchemy connection string for five relational tables
MAX_CONCURRENCY	1	Sequential worker count preventing cross-job CPU cache/CFS interference

Setting Key	Default Value	System Component Governed
JOB_QUEUE_CAPACITY	50	Bounded asyncio.Queue capacity returning HTTP 503 when saturated
STORAGE_DIR	/tmp/sandboxed_storage	Parent directory for per-execution ephemeral exec_<id> workspaces
DEFAULT_ / MAX_WALL_TIME	5.0 s / 60.0 s	Fallback and ceiling limits referenced by API configuration
DEFAULT_ / MAX_MEMORY_MB	128 MiB / 1024 MiB	Backend configuration memory baseline and cap
BWRAP_PATH / CGROUP_ROOT	/usr/bin/bwrap, /sys/fs/cgroup	Target Linux binary and unified cgroups v2 hierarchy root
CGROUP_SLICE	sandboxed.slice	Delegated cgroup parent folder owned by the unprivileged service user

4.5 Deployment Provisioning and Application Hardening

Least-privilege deployment separates one-time root setup from daily execution. Operators run scripts/setup_ubuntu.sh with sudo exactly once to install apt packages (bubblewrap, libseccomp-dev, libseccomp2, python3-venv, python3-pip, curl, git, and Node.js 20), enable +cpu +memory +pids in /sys/fs/cgroup/cgroup.subtree_control, create /sys/fs/cgroup/sandboxed.slice owned by the real operator, chmod /tmp/sandboxed_storage to 770, and build both virtual environments under the operator's user UID.

Once provisioned, neither Uvicorn nor the React frontend uses root privileges. Inside the API service, PolicyRepository blocks deletion of standard and strict baseline profiles with ValueError (mapped to HTTP 400), ExecutionService records policy snapshots alongside SHA-256 hashes, and FastAPI lifespan startup reconciles any orphaned QUEUED, PREPARING, or RUNNING rows from abrupt restarts as CANCELLED (INTERRUPTED).

5. System Architecture and Workflow

5.1 End-to-End Job Submission and Policy Resolution

Execution starts at POST /api/executions with UTF-8 Python code and either a saved policy_id or an inline custom_policy block. ExecutionService queries PolicyRepository to resolve the effective ExecutionPolicy (falling back to DEFAULT_POLICIES when needed or defaulting to standard), hashes the source text with SHA-256, allocates a UUID execution identifier, persists a QUEUED ExecutionRecord carrying a JSON policy snapshot, and writes an EXECUTION_SUBMITTED row into audit_events.

Next, ExecutionManager.submit_job() pushes an ExecutionJob into an asyncio.Queue limited to 50 pending entries. When all 50 slots are full, submit_job() raises RuntimeError so the HTTP router responds cleanly with 503 Service Unavailable. One worker coroutine drains the queue sequentially so parallel experiments never fight for host cores or corrupt timing benchmarks.

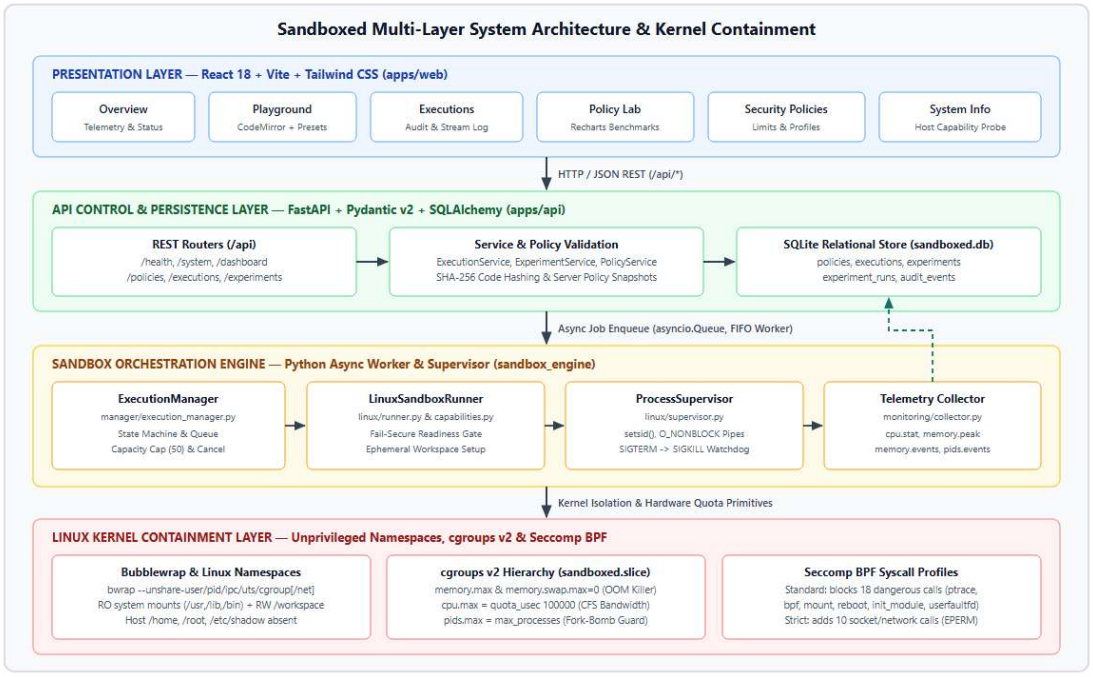


Figure 5.1: Sandboxed Multi-Layer System Architecture and Kernel Containment

Description: Four-tier architectural layout depicting REST calls from the React 18 UI into FastAPI services and SQLite, asynchronous worker dispatch in sandbox_engine, and physical containment via bubblewrap namespaces, cgroups v2 controllers, and seccomp profile definitions.

5.2 Host Capability Probing and Fail-Secure Gating

Upon dequeueing a job not marked cancelled, the worker switches job state to PREPARING and invokes LinuxSandboxRunner.run_program() inside asyncio.to_thread(). Running blocking subprocess supervision on a worker thread keeps the FastAPI asyncio event loop responsive to UI status polls and cancellation POSTs.

Before touching temporary folders or cgroups, run_program() invokes detect_capabilities(). If host checks indicate Windows, absent bwrap, disabled user namespaces, legacy cgroups v1, missing cpu/memory controllers, or unwritable slice permissions, SandboxCapabilityError aborts launch immediately. ExecutionManager catches that exception, transitions the state to FAILED with termination_reason=PREPARATION_ERROR, and logs exact diagnostic reasons into stderr and error_summary.

5.3 Per-Execution Cgroup Allocation and Bubblewrap Construction

When capability checks succeed, LinuxSandboxRunner creates /tmp/sandboxed_storage/exec_<execution_id>, writes the payload as script.py, and instructs CgroupV2Manager to prepare exec_<execution_id> beneath /sys/fs/cgroup/sandboxed.slice. That folder receives byte limits in memory.max, zero in memory.swap.max, quota microseconds over 100000 in cpu.max, and task limits in pids.max before the guest interpreter starts.

Notice how child-to-cgroup race windows are eliminated. If a short script allocated memory during the first millisecond after execve before the parent API thread moved its PID into cgroup.procs, kernel OOM limits could miss the burst. Sandboxed writes the child PID into cgroup.procs inside POSIX preexec_fn right after fork() and reinforces that assignment from the parent process right after Popen returns.

BwrapCommandBuilder then produces an argument array for execve without invoking a shell. It unshares user, IPC, PID, UTS, and cgroup namespaces, adds --unshare-net by default, enables --die-with-parent and --new-session, mounts virtual /proc, /dev, and tmpfs /tmp, bind-mounts existing host runtime folders (/usr, /lib, /lib64, /bin, /sbin, /etc/alternatives, /etc/ld.so.cache) read-only, masks /etc under an empty tmpfs when STRICT_RO is chosen, and exposes only the isolated workspace at /workspace.

5.4 Supervised Process Execution and Watchdog Escalation

Once RUNNING starts, ProcessSupervisor launches bwrap with subprocess.Popen and the preexec_fn callback calling os.setsid() alongside cgroup attachment. Establishing a fresh session and process group leadership is vital: ordinary proc.terminate() calls only signal the outer bubblewrap binary, leaving forked grandchildren detached. Setting O_NONBLOCK on stdout and stderr descriptors via fcntl prevents readline or communicate deadlocks when untrusted loops flood kernel pipe buffers.

Inside a 20 ms polling loop, the supervisor watches user cancellation checks, monotonic elapsed runtime against policy.wall_time_limit_sec, and independent 4,096-byte reads from stdout and stderr up to policy.max_output_bytes. Whenever bytes exceed the configured cap, extra slices are dropped cleanly and output_truncated switches to True. If cancellation arrives or elapsed time exceeds the wall deadline, _kill_process_group() signals -pgid with SIGTERM, waits 0.3 seconds for orderly exit, and sends SIGKILL.

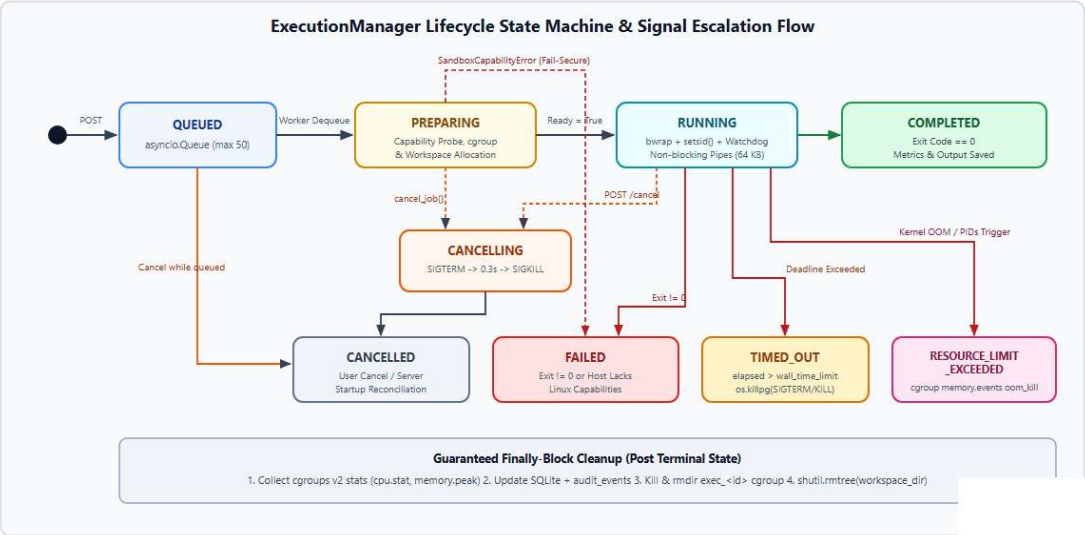


Figure 5.2: ExecutionManager Lifecycle State Machine and Signal Escalation Flow

Description: State transition graph illustrating job movement across QUEUED, PREPARING, RUNNING, and CANCELLING into five terminal states (COMPLETED, FAILED, TIMED_OUT, CANCELLED, RESOURCE_LIMIT_EXCEEDED) with teardown.

5.5 Telemetry Harvesting, Cleanup, and Lifecycle Finalisation

Order of operations during teardown is critical. Kernel counters live inside the transient cgroup folder; deleting the cgroup first erases peak memory and OOM evidence. ProcessSupervisor therefore drains any final bytes from stdout and stderr pipes, reads cpu.stat, memory.peak, memory.current, memory.events, and pids.events via

CgroupV2Manager.collect_metrics(), and sets termination_reason to RESOURCE_LIMIT_EXCEEDED whenever kernel oom_kill or oom counters are non-zero.

Only after metrics return to Python memory does LinuxSandboxRunner execute its finally block, triggering cgroup.kill (falling back to iterating PIDs inside cgroup.procs with SIGKILL), removing the child cgroup directory after brief kernel draining, and deleting /tmp/sandboxed_storage/exec_<execution_id>.

Table 5.1: Execution Lifecycle Stages and Component Ownership

Lifecycle Stage	Owning Module / Class	Key Kernel / Persistence Action
1. Intake & Hashing	ExecutionService (execution_service.py)	Resolves policy, computes SHA-256, persists QUEUED row and audit event
2. Queue & Dispatch	ExecutionManager (execution_manager.py)	Enqueues ExecutionJob into bounded asyncio.Queue (capacity 50)
3. Capability Gate	detect_capabilities (capabilities.py)	Verifies Linux OS, bwrap, userns, cgroups v2 controllers, and slice write access
4. Quota & Mounts	CgroupV2Manager & BwrapCommandBuilder	Writes memory.max/cpu.max/pids.max and builds unprivileged bwrap CLI vector
5. Supervision	ProcessSupervisor (supervisor.py)	Establishes setsid(), polls non-blocking pipes, enforces monotonic deadline
6. Telemetry & Purge	collect_cgroup_v2_metrics & cleanup_cgroup	Reads cpu.stat/memory.peak/oom_kill, updates SQLite, deletes cgroup and /workspace

5.6 Relational Persistence and Immutable Policy Snapshots

Five SQLAlchemy ORM models in apps/api/app/models/entities.py map application history onto SQLite: PolicyRecord (policies), ExecutionRecord (executions), ExperimentRecord (experiments), ExperimentRunRecord (experiment_runs), and AuditEventRecord (audit_events). Indexed columns on source_hash, state, submitted_at, audit_events.execution_id, experiment_id, event_type, and timestamp keep dashboard filters fast.

Notice one deliberate referential design choice illustrated in Figure 5.3. Between experiments and experiment_runs, SQLAlchemy enforces a strict SQL ForeignKey with ON DELETE CASCADE (cascade='all, delete-orphan') because child benchmark runs have no independent meaning if their parent experiment is deleted. By contrast, executions and audit_events store logical string references to policy_id and execution_id alongside a frozen JSON policy_snapshot string. Editing or removing a custom policy later never invalidates or orphans historical execution telemetry.

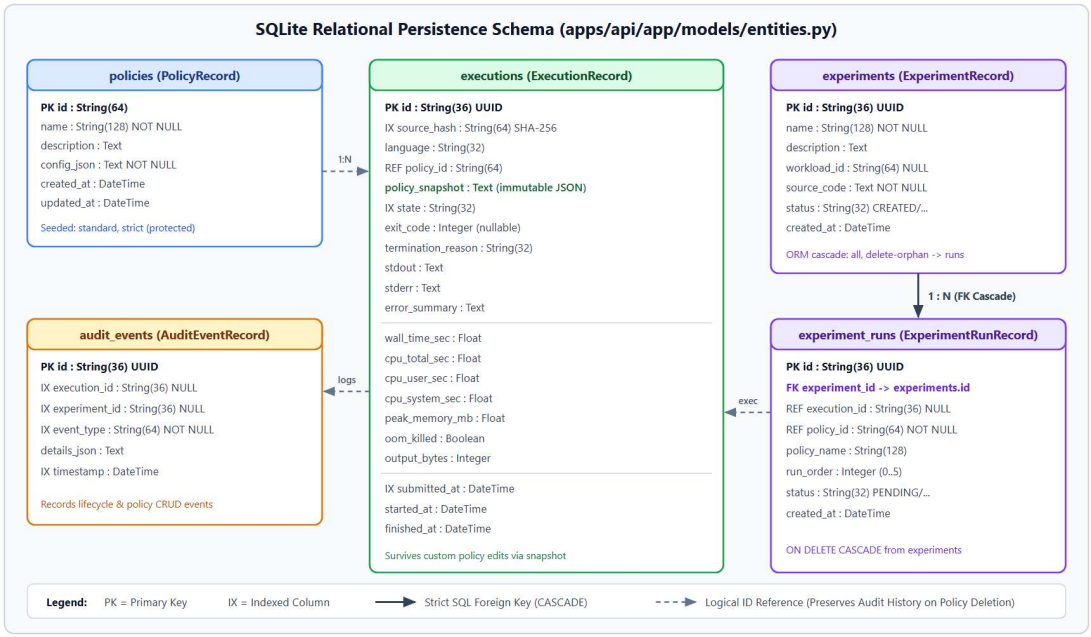


Figure 5.3: SQLite Relational Persistence Schema (apps/api/app/models/entities.py)

Description: Entity-relationship diagram detailing all five SQLite tables, primary keys, indexed query columns, CASCADE foreign key linkage from experiments to experiment_runs, and immutable policy_snapshot retention.

Table 5.2: SQLite Database Tables, Keys, and Index Design

Table / ORM Class	Primary Key	Indexed / FK Columns	Stored Data & Retention Semantics
policies (PolicyRecord)	id : String(64)	None (small profile table)	Custom and seeded policies JSON; standard/strict protected from DELETE
executions (ExecutionRecord)	id : String(36)	IX source_hash, state, submitted_at	Source hash, streams, 7 telemetry counters, timestamps, policy_snapshot JSON
experiments (ExperimentRecord)	id : String(36)	ORM runs relationship	Benchmark name, workload_id, source_code, status CREATED/RUNNING/COMPLETED
experiment_runs (ExperimentRunRecord)	id : String(36)	FK experiment_id CASCADE	Ordered policy_id runs linked logically to produced execution_id UUIDs
audit_events (AuditEventRecord)	id : String(36)	IX execution_id, experiment_id, event_type, timestamp	Append-only operational trail for submissions, job state transitions, policy CRUD

5.7 Asynchronous Callback Synchronisation and Startup Recovery

ExecutionManager intentionally stays free of SQLAlchemy imports. Wiring between engine transitions and SQLite happens exclusively inside apps/api/app/api/deps.py via an async callback _handle_job_update(job). Every time a job shifts state (QUEUED to PREPARING, RUNNING, and its terminal state), that callback opens a short-lived SessionLocal database

session, runs ExecutionRepository.update_from_job(job), and appends a matching JOB_<STATE> event to AuditRepository inside a guaranteed try/finally session close.

Restart resilience completes the lifecycle loop inside apps/api/app/main.py. When Uvicorn boots FastAPI, the lifespan context manager creates schema tables, seeds default policies idempotently, and queries for any ExecutionRecord still marked QUEUED, PREPARING, or RUNNING from a crashed prior session. Those stale records transition automatically to CANCELLED with termination_reason='INTERRUPTED' before worker tasks start.

6. Modules of the Project

6.1 Policy Domain Models and Default Profiles

The schema module sandbox_engine/policies/models.py establishes two isolation enums (FilesystemPolicy with values isolated_ro_root, minimal, and strict_ro; SyscallPolicy with standard, strict, and disabled) alongside the ExecutionPolicy Pydantic model. Every numeric parameter includes declarative range metadata: wall_time_limit_sec runs from 0.1 to 60.0, memory_limit_mb spans 16 to 2048 MiB, cpu_quota_pct spans 10 to 100 percent, max_processes caps tasks between 1 and 128, max_output_bytes restricts capture between 1,024 and 1,048,576 bytes, and temp_storage_mb ranges from 8 to 512 MiB.

Rather than forcing instructors to construct every limit manually before running an initial class demonstration, DEFAULT_POLICIES supplies six pre-tuned templates summarized in Table 6.1. Two baseline identifiers, standard and strict, carry explicit deletion locks inside PolicyRepository so HTTP clients can examine or benchmark them without accidentally removing required defaults.

Table 6.1: Default Security and Resource Policy Profiles (DEFAULT_POLICIES)

Policy ID	Wall Time	RAM Limit	CPU Quota	Max PIDs	Filesystem / Syscalls
standard	5.0 s	128 MiB	50% CFS	32	isolated_ro_root / standard
strict	3.0 s	64 MiB	30% CFS	8	strict_ro / strict (32 KiB out)
cpu_constrained	5.0 s	128 MiB	20% CFS	16	isolated_ro_root / standard
memory_constrained	5.0 s	32 MiB	50% CFS	16	isolated_ro_root / standard
network_blocked	5.0 s	128 MiB	50% CFS	32	isolated_ro_root (--unshare-net)
permissive	10.0 s	256 MiB	100% CFS	64	isolated_ro_root (128 KiB out)

6.2 Built-in Academic Benchmark Workloads

The second half of sandbox_engine/policies/models.py exports BUILTIN_WORKLOADS: eight deterministic Python scripts modeled via BuiltinWorkload (id, title, description, category, expected_outcome, code). Each workload probes one specific operating system mechanism, ranging from arithmetic sanity checks to controlled resource exhaustion and filesystem escape attempts, so Policy Lab experiments stay reproducible across semesters.

Inspection of the workload implementations clarifies how they exercise Linux limits. Specifically, cpu_intensive invokes a naive O(n * sqrt(n)) prime divisor loop up to 25,000 that saturates single-core scheduling, making 100 percent versus 20 percent CFS throttling immediately visible. Workload memory_allocation allocates ten sequential 5 MiB bytearray

blocks separated by 50 ms sleeps up to 50 MiB total, succeeding beneath the 128 MiB standard ceiling while crossing memory.max deterministically on the 32 MiB profile. Workloads fork_limit_test and output_limit_test spawn up to 50 multiprocessing children or write 5,000 log lines to verify pids.max rejection and non-blocking stdout truncation.

Table 6.2: Built-in Operating Systems Test Workloads (BUILTIN_WORKLOADS)

Workload ID	OS Category	Measured Stress Pattern	Expected Sandbox Behaviour
normal_output	Baseline	Sums i^2 over 0..9999 and prints runtime	Exit 0, status COMPLETED under 1 s
cpu_intensive	Resource Enforcement	Searches primes up to 25,000 in tight loops	Completes or hits TIMED_OUT under low CPU
memory_allocation	Resource Enforcement	Allocates 50 MiB via ten 5 MiB bytearrays	COMPLETED on 128 MiB; OOM on 32 MiB
deadline_timeout	Process Supervision	Sleeps 19 s printing 1-second tick lines	Supervisor kills group at wall_time_limit_sec
file_escape_attempt	Security & Isolation	Probes /etc/shadow, /root, ~/.ssh, bash_history	FileNotFoundError or PermissionError
network_attempt	Security & Isolation	Connects TCP socket to 8.8.8.8:53 (2s timeout)	OSError / ENETUNREACH inside net namespace
output_limit_test	Process Supervision	Prints 5,000 log lines rapidly to stdout	Truncated cleanly at max_output_bytes
fork_limit_test	Resource Enforcement	Attempts spawning 50 child processes	Halted when hitting cgroups v2 pids.max

6.3 Host Capability Detector

Host probing lives inside sandbox_engine/linux/capabilities.py through detect_capabilities() and the 16-field EngineCapabilityReport schema. When invoked on Windows or macOS, it immediately populates is_linux=False and ready=False with explanatory strings instead of touching nonexistent /proc paths. When invoked on Linux, it reads PRETTY_NAME from /etc/os-release, checks bwrap --version with a 2-second timeout, inspects /proc/sys/kernel/unprivileged_userns_clone and /proc/sys/user/max_user_namespaces, reads /sys/fs/cgroup/cgroup.controllers, tests mkdir/os.access write privileges inside sandboxed.slice (or falls back to user.slice/user-<uid>.slice), and verifies CONFIG_SECCOMP in /boot/config-<kernel>.

All sixteen structured fields serialized to GET /api/system/capabilities preserve exact machine truth: ready, os_name, os_release, kernel_version, is_linux, bwrap_available, bwrap_path, bwrap_version, user_namespaces_supported, cgroups_v2_mounted, cgroups_controllers_available, cgroups_delegation_writable, cgroups_path, seccomp_supported, python_version, and issues. Readiness evaluates True only when Linux

is confirmed alongside bwrap presence, user namespace support, unified cgroups v2 mounting, writable slice delegation, and active cpu and memory controllers.

6.4 Bubblewrap Command Builder

BwrapCommandBuilder in sandbox_engine/linux/bwrap_builder.py turns policy flags and a workspace path into an argument vector executed directly via execve. It always adds --unshare-user, --unshare-ipc, --unshare-pid, --unshare-uts, and --unshare-cgroup, selects --unshare-net unless policy.network_enabled is True, and binds --die-with-parent plus --new-session so background subshells cannot persist if the parent controller exits.

Virtual mounts cover --proc /proc, --dev /dev, and an isolated --tmpfs /tmp. Read-only system runtime paths (/usr, /lib, /lib64, /bin, /sbin, /etc/alternatives, /etc/ld.so.cache) are attached with --ro-bind only if present on disk. Under FilesystemPolicy.STRICT_RO, an empty --tmpfs /etc hides /etc/resolv.conf entirely; otherwise /etc/resolv.conf attaches read-only. Finally, the unique job directory binds rw at /workspace with --chdir /workspace, optional --seccomp <fd> attaches when supplied, and python3 runs -u /workspace/script.py.

6.5 Cgroups v2 Resource Manager

CgroupV2Manager in sandbox_engine/linux/cgroup_v2.py creates dedicated child directories named exec_<execution_id> beneath /sys/fs/cgroup/sandboxed.slice. Writing policy.memory_limit_mb * 1024 * 1024 to memory.max sets the RAM cap, while writing 0 to memory.swap.max closes swap escape routes. CPU throttling fixes the CFS period at 100,000 us and calculates quota_usec = int((policy.cpu_quota_pct / 100.0) * 100000) inside cpu.max, while pids.max restricts fork growth.

Parent slice initialization inside setup_slice() writes '+cpu +memory +pids' into sandboxed.slice/cgroup.subtree_control so child directories automatically inherit active resource files. Teardown inside cleanup_cgroup() defends against slow kernel accounting drains. On Linux 5.14+, it writes 1 to cgroup.kill to stop every task in one atomic kernel call, scans any remaining entries in cgroup.procs with os.kill(pid, signal.SIGKILL), waits 0.05 seconds before rmdir(), and performs one 0.2-second retry if the kernel still reports EBUSY.

6.6 Seccomp Syscall Profile Definitions

Syscall containment profiles live in sandbox_engine/linux/seccomp_filter.py. DANGEROUS_SYSCALLS lists 18 high-privilege or exploit-relevant system calls: ptrace, chroot, reboot, kexec_load, kexec_file_load, mount, umount2, swapon, swapoff, init_module,

finit_module, delete_module, bpf, userfaultfd, perf_event_open, settimeofday, clock_settime, and acct. NETWORK_SYSCALLS appends 10 socket primitives: socket, connect, bind, listen, accept, accept4, sendto, recvfrom, sendmsg, and recvmsg.

Function get_syscall_policy_definition() maps DISABLED (0 blocked calls), STANDARD (18 blocked calls), and STRICT (all 28 deduplicated syscalls blocked) into a typed SyscallFilterDefinition model validated by five unit tests. Blocking userfaultfd and perf_event_open closes common kernel heap-spraying and side-channel gadgets, while denying bpf and init_module blocks kernel bytecode loading. While BwrapCommandBuilder already exposes an optional seccomp_bpf_fd flag for --seccomp FD injection, LinuxSandboxRunner currently relies on namespaces and cgroups v2 at runtime, keeping BPF bytecode serialization ready as a modular extension.

6.7 Execution Runner and Supervision Engine

LinuxSandboxRunner in sandbox_engine/linux/runner.py acts as the single runtime coordinator inside the engine. Its run_program() method runs detect_capabilities(), creates /tmp/sandboxed_storage/exec_<id>, writes script.py in UTF-8, prepares the child cgroup when writable, constructs bwrap_cmd, and delegates execution to ProcessSupervisor.run_supervised(). A guaranteed finally block removes both the cgroup slice and temporary workspace even when exceptions fire.

ProcessSupervisor in sandbox_engine/linux/supervisor.py establishes session isolation using os.setsid() inside _preexec_child(), sets O_NONBLOCK on stdout/stderr pipes via fcntl, reads up to 4,096 bytes per cycle under policy.max_output_bytes, checks cancel_check() and monotonic deadlines every 20 ms, and maps non-zero kernel oom counters cleanly to TerminationReason.RESOURCE_LIMIT_EXCEEDED. Even if non-blocking reads catch partial output while exiting, a dedicated post-exit drain step collects trailing buffered bytes within the configured output byte budget before UTF-8 decoding with errors='replace'.

6.8 Asynchronous Queue Manager and State Machine

ExecutionManager in sandbox_engine/manager/execution_manager.py bounds pending jobs using asyncio.Queue(maxsize=max_queue_size) and tracks live state across nine ExecutionState literals: QUEUED, PREPARING, RUNNING, CANCELLING, COMPLETED, FAILED, TIMED_OUT, CANCELLED, and RESOURCE_LIMIT_EXCEEDED. Calling cancel_job() on a queued item immediately flips

it to CANCELLED and suppresses execution on dequeue, while calling cancel_job() on an active job sets CANCELLING and halts the supervisor via its cancel_check closure.

Health diagnostics can inspect live worker metrics at any moment through ExecutionManager.get_stats(), which reports queue_depth, active_workers, total_tracked_jobs, and running_count. Finished jobs in COMPLETED, FAILED, TIMED_OUT, CANCELLED, or RESOURCE_LIMIT_EXCEEDED reject duplicate cancellation requests by returning False so clients receive deterministic HTTP 400 feedback rather than corrupting historical state.

6.9 Cgroups v2 Telemetry Collector

Telemetry parsing lives inside sandbox_engine/monitoring/collector.py, separate from lifecycle code so parser unit tests work across synthetic temp folders on Windows. ExecutionMetrics defines 14 typed attributes: wall_time_sec, cpu_time_user_sec, cpu_time_system_sec, cpu_usage_total_sec, peak_memory_bytes, peak_memory_mb, current_memory_bytes, oom_killed, oom_events_count, pids_current, pids_peak, pids_max_events, output_bytes, and truncated_output. Kernel microsecond metrics in cpu.stat divide by 1,000,000.0, while read_int_file() maps unbounded cgroup sentinel strings such as 'max' safely to -1.

Helper function parse_key_value_file() tolerates unreadable or missing pseudo-files across older kernel builds by returning empty dictionaries instead of raising FileNotFoundError mid-collection. Whenever oom_kill or oom inside memory.events exceeds zero, collect_cgroup_v2_metrics() flags oom_killed=True back to the supervisor so physical out-of-memory kills never masquerade as ordinary Python nonzero exits.

6.10 FastAPI Data Repositories, Services, and REST Endpoints

Persistent storage access splits cleanly across four SQLAlchemy repository classes in apps/api/app/repositories/. PolicyRepository exposes seed_defaults(), list_all(), get_by_id(), save(), and delete() while raising ValueError on standard or strict deletions. ExecutionRepository implements create(), get_by_id(), delete(), paginated list_executions() filtered by optional state, update_from_job(), and grouped SQL COUNT queries via get_summary_counts(). ExperimentRepository manages parent experiments and ordered runs, while AuditRepository writes timestamped JSON detail events capped by query limit.

Business rules reside in PolicyService, ExecutionService, and ExperimentService. In Policy Lab benchmarks, ExperimentService.run_experiment() iterates sorted runs serially and polls

SQLite every 0.5 seconds until each execution reaches a terminal status. After completion, get_results() invokes _interpret_results() to synthesize automated OS findings noting verified OOM kills, deadline timeouts, and CFS timing variance whenever duration spread exceeds 0.05 seconds across successful runs. Table 6.3 lists all HTTP routes.

Table 6.3: Complete FastAPI REST Endpoint Catalogue

HTTP Method & Path	Router Module	Functional Responsibility
GET /api/health	health.py	Returns service health, API version 1.0.0, and current UTC timestamp
GET /api/system/capabilities	system.py	Invokes detect_capabilities() and returns host kernel/bwrap/cgroup state
GET /api/system/workloads	system.py	Returns all 8 BuiltinWorkload templates with source code and metadata
GET /api/dashboard/summary	dashboard.py	Aggregates DB counts by state, queue depth, readiness, and 10 latest runs
GET / POST /api/policies	policies.py	Lists seeded/custom policies or creates validated custom ExecutionPolicy rows
GET / PATCH / DELETE /api/policies/{id}	policies.py	Inspects, edits, or deletes custom policies (blocks standard/strict delete)
POST / GET /api/executions	executions.py	Validates code, resolves policy snapshot, queues run, or lists paginated runs
GET /api/executions/{id}, /output, /metrics	executions.py	Retrieves detail record, stdout/stderr captures, and cgroups v2 counters
POST /api/executions/{id}/cancel	executions.py	Signals cancellation on queued or running jobs and logs audit event
POST / GET /api/experiments	experiments.py	Creates comparative Policy Lab benchmark records or lists experiment runs
POST /api/experiments/{id}/run, GET /results	experiments.py	Dispatches background sequential benchmark and returns interpreted comparison

6.11 React Presentation Workspace

The browser client under apps/web/src/ wraps seven routed views inside AppLayout.jsx. On every route transition, AppLayout queries GET /api/system/capabilities so the top header pill renders a truthful green Ready (Linux) indicator on configured Ubuntu systems or an amber Host: Non-Linux / Missing Setup alert on Windows or unprepared Linux hosts. Code editing uses CodeMirror with Python syntax highlighting, while comparative Policy Lab measurements render through Recharts bar plots. Table 6.4 details all seven pages.

Table 6.4: React Frontend Views and Functional Responsibilities

Route & Component	Source Module	Primary Widgets & Data	User Workflow
/ (Overview)	Overview.jsx	Summary KPI cards, Recharts bar plot, host probe strip	Monitor global job counts and latest runs

Route & Component	Source Module	Primary Widgets & Data	User Workflow
/playground (Playground)	Playground.jsx	CodeMirror editor, 8 workload presets, .py upload, logs	Edit/submit scripts with live output tabs
/executions (Executions)	Executions.jsx	State filter bar, table, STDOUT/STDERR/METRICS/POLICY tabs	Audit historical jobs and policy snapshots
/policy-lab (Policy Lab)	PolicyLab.jsx	Workload picker, policy checkboxes, comparison table & chart	Run side-by-side OS policy experiments
/policies (Policies)	Policies.jsx	6 Core-locked cards + custom policy builder modal	Inspect limits or create/delete custom rules
/system (System Info)	SystemInfo.jsx	Capability table, bwrap/cgroup/seccomp checks, issues list	Diagnose Ubuntu host readiness or gaps
/settings (Settings)	Settings.jsx	Read-only queue (50), concurrency (1), 64 KiB, DB cards	Review locked runtime safety ceilings

7. Implementation and Source Code

7.1 Host Capability Probing and Honest Non-Linux Reporting

Before touching namespaces or cgroups, detect_capabilities() evaluates platform.system(). Returning early when running on Windows or macOS avoids spurious filesystem warnings while populating clear remediation issues for the frontend status banner. Listing 7.1 shows this platform guard and the Linux user-namespace probe.

Listing 7.1: Host OS Gate and User Namespace Verification

```
current_os = platform.system()
is_linux = (current_os == "Linux")
if not is_linux:
    issues.append(f"Host OS is '{current_os}'. Requires Ubuntu Linux.")
    return EngineCapabilityReport(
        ready=False, os name=current os, kernel version=kernel ver,
        is_linux=False, python_version=py_ver, issues=issues,
    )
userns_path = Path("/proc/sys/kernel/unprivileged_userns_clone")
max userns path = Path("/proc/sys/user/max user namespaces")
if userns path.is file():
    userns_supported = (userns_path.read_text().strip() == "1")
elif max_userns_path.is_file():
    userns_supported = (int(max_userns_path.read_text().strip()) > 0)
```

Source: sandbox_engine/linux/capabilities.py (excerpt from detect_capabilities).

Notice how the probe branches across kernel distributions: Debian-derived kernels expose unprivileged_userns_clone whereas newer mainline kernels govern creation via max_user_namespaces. Checking both sysctl paths prevents false negatives across kernel 5.15 and 6.x releases.

7.2 Policy Constraints and Validation Rules

Declaring bounds directly on the Pydantic domain schema guarantees that invalid wall times, oversized memory requests, or out-of-range PID caps fail identically whether parsed from JSON snapshots or REST payloads.

Listing 7.2: ExecutionPolicy Resource Envelope Definitions

```
class FilesystemPolicy(str, Enum):
    ISOLATED_RO_ROOT = "isolated_ro_root"
    MINIMAL = "minimal"
    STRICT RO = "strict ro"


class ExecutionPolicy(BaseModel):
    id: str = Field(...)
    name: str = Field(..., min_length=2, max_length=64)
    wall time limit sec: float = Field(default=5.0, ge=0.1, le=60.0)
    memory limit mb: int = Field(default=128, ge=16, le=2048)
    cpu quota pct: int = Field(default=50, ge=10, le=100)
    max_processes: int = Field(default=32, ge=1, le=128)
    max_output_bytes: int = Field(default=65536, ge=1024, le=1048576)
    network_enabled: bool = Field(default=False)
```

Source: sandbox_engine/policies/models.py (FilesystemPolicy and ExecutionPolicy).

Defaulting network_enabled to False follows fail-closed defaults: an omitted parameter always keeps the network namespace disconnected unless an authorized experiment toggles egress explicitly.

7.3 Bubblewrap Namespace and Mount Assembly

BwrapCommandBuilder constructs a deterministic argument array with zero shell interpretation. Notice how STRICT_RO swaps /etc with an ephemeral empty tmpfs so DNS configuration files become unreachable inside the jail.

Listing 7.3: Namespace Unsharing and Read-Only Mount Synthesis

```
cmd: List[str] = [self.bwrap path]
cmd.extend([
    "--unshare-user", "--unshare-ipc", "--unshare-pid",
    "--unshare-uts", "--unshare-cgroup",
])
cmd.append("--unshare-net" if not policy.network enabled else "--share-net")
cmd.extend(["--die-with-parent", "--new-session"])
cmd.extend(["--proc", "/proc", "--dev", "/dev", "--tmpfs", "/tmp"])
for ro_path in ["/usr", "/lib", "/lib64", "/bin", "/sbin"]:
    if Path(ro_path).exists():
        cmd.extend(["--ro-bind", ro_path, ro_path])
if policy.filesystem policy == FilesystemPolicy.STRICT RO:
    cmd.extend(["--tmpfs", "/etc"])
cmd.extend(["--bind", str(sandbox_workspace.resolve()), "/workspace"])
cmd.extend(["--chdir", "/workspace", interpreter, "-u",
f"/workspace/{script_filename}"])
```

Source: sandbox_engine/linux/bwrap_builder.py (excerpt from BwrapCommandBuilder.build_command).

Checking Path(ro_path).exists() before appending --ro-bind prevents startup crashes across distributions with merged /usr architectures, while python3 -u disables stdout buffering so timed-out scripts still flush logs.

7.4 Cgroups v2 Quota Configuration and Teardown

Setting memory.swap.max to zero ensures guest scripts cannot bypass RAM limits by swapping into host disk. CFS quotas compute microseconds relative to a fixed 100,000 us window before cleanup_cgroup() tears down tasks.

Listing 7.4: Cgroups v2 Quota Setup and Atomic Cleanup

```
exec_cgroup = self.slice_dir / f"exec_{execution_id}"
exec_cgroup.mkdir(parents=True, exist_ok=True)
mem limit bytes = policy.memory limit mb * 1024 * 1024
(exec cgroup / "memory.max").write text(f"{mem limit bytes}\n", encoding="utf-8")
if (exec cgroup / "memory.swap.max").is file():
    (exec_cgroup / "memory.swap.max").write_text("0\n", encoding="utf-8")
period_usec = 100000
quota_usec = int((policy.cpu_quota_pct / 100.0) * period_usec)
(exec cgroup / "cpu.max").write text(f"{quota usec} {period usec}\n", encoding="utf-8")
(exec_cgroup / "pids.max").write_text(f"{policy.max_processes}\n", encoding="utf-8")
...
if (exec_cgroup / "cgroup.kill").is_file():
    (exec_cgroup / "cgroup.kill").write_text("1\n", encoding="utf-8")
time.sleep(0.05)
exec_cgroup.rmdir()
```

Source: sandbox_engine/linux/cgroup_v2.py (create_execution_cgroup and cleanup_cgroup).

The short 50 ms delay before rmdir() accounts for kernel task reaping: Linux refuses to unlink a cgroup directory until the final zombie descriptor clears cgroup.procs, and cleanup_cgroup() retries 0.2 s later if busy.

7.5 Seccomp Syscall Denylist Profiles

Declarative filtering tables isolate dangerous system-level calls separately from network socket calls, allowing unit tests to audit both Standard (18 denied syscalls) and Strict (28 denied syscalls) compositions cleanly.

Listing 7.5: Declarative Seccomp Syscall Profile Resolution

```
def get syscall policy definition(
    policy: SyscallPolicy,
) -> SyscallFilterDefinition:
    if policy == SyscallPolicy.DISABLED:
        return SyscallFilterDefinition(
            policy=policy, default action="ALLOW", blocked syscalls=[],
            description="Syscall filtering disabled (namespaces only).",
        )
    elif policy == SyscallPolicy.STRICT:
        blocked = sorted(list(set(DANGEROUS_SYSCALLS + NETWORK_SYSCALLS)))
        return SyscallFilterDefinition(
            policy=policy, default action="ALLOW", blocked syscalls=blocked,
            description="Strict: blocks dangerous calls and network sockets.",
        )
    return SyscallFilterDefinition(
        policy=SyscallPolicy.STANDARD, default action="ALLOW",
        blocked syscalls=sorted(DANGEROUS SYSCALLS),
        description="Standard: rejects privileged administrative syscalls.",
    )
```

Source: sandbox_engine/linux/seccomp_filter.py (get_syscall_policy_definition).

Using set unions in STRICT mode deduplicates call names cleanly while returning sorted lists for deterministic UI display and equality assertions during regression testing.

7.6 Runner Orchestration and Guaranteed Cleanup

LinuxSandboxRunner coordinates capability verification, ephemeral file creation, cgroup provisioning, and bwrap command supervision inside a single try/finally block so neither aborted scripts nor exceptions leave orphaned workspaces on disk.

Listing 7.6: Fail-Secure Gate and Deterministic Teardown in LinuxSandboxRunner

```
if not force_bypass_check:
    cap_report = detect_capabilities(str(self.cgroup_manager.cgroup_root))
    if not cap_report.ready:
        raise SandboxCapabilityError(cap report)
workspace dir = self.storage dir / f"exec {execution id}"
workspace_dir.mkdir(parents=True, exist_ok=True)
script_path = workspace_dir / "script.py"
script_path.write_text(code, encoding="utf-8")
try:
    self.cgroup manager.setup slice()
    exec cgroup = self.cgroup manager.create execution cgroup(execution id, policy)
    bwrap_cmd = self.bwrap_builder.build_command(policy=policy,
sandbox_workspace=workspace_dir)
    return self.supervisor.run_supervised(cmd=bwrap_cmd, policy=policy, ...)
finally:
    if exec cgroup and exec cgroup.exists():
        self.cgroup manager.cleanup cgroup(exec cgroup)
    shutil.rmtree(workspace_dir, ignore_errors=True)
```

Source: sandbox_engine/linux/runner.py (abridged from LinuxSandboxRunner.run_program).

Placing detect_capabilities() ahead of filesystem creation guarantees that failed readiness probes leave zero temporary directories behind in /tmp/sandboxed_storage.

7.7 Non-Blocking Supervision and Watchdog Escalation

ProcessSupervisor establishes a new session via os.setsid() before exec, configures O_NONBLOCK on stdout and stderr descriptors, checks monotonic deadlines every 20 ms, and terminates stubborn descendant groups using SIGTERM followed 0.3 seconds later by SIGKILL.

Listing 7.7: Non-Blocking Supervision Loop and Signal Escalation

```
def _preexec_child():
    os.setsid()
    if cgroup_path:
        procs_file = cgroup_path / "cgroup.procs"
        if procs file.is file():
            with open(procs_file, "w") as f:
                f.write(f"{os.getpid()}\n")
...
if elapsed > timeout sec:
    termination reason = TerminationReason.TIMED OUT
    self._kill_process_group(proc)
    break
...
def _kill_process_group(self, proc: subprocess.Popen) -> None:
    if proc.poll() is not None: return
    if os.name != "nt":
        os.killpg(os.getpgid(proc.pid), signal.SIGTERM)
        time.sleep(0.3)
        if proc.poll() is None:
            os.killpg(os.getpgid(proc.pid), signal.SIGKILL)
```

Source: sandbox_engine/linux/supervisor.py (excerpt from run_supervised and _kill_process_group).

Signalling os.killpg(os.getpgid(proc.pid), ...) reaches every descendant created inside the session. Using a 0.3-second SIGTERM window gives clean runtimes a moment to unwind before unavoidable SIGKILL termination.

7.8 Kernel Counter Parsing and Telemetry Conversion

Once execution stops and before cleanup_cgroup() executes, collect_cgroup_v2_metrics() reads cpu.stat, memory.peak, memory.events, and pids.events directly from the slice folder and converts microseconds into floating-point seconds.

Listing 7.8: Harvesting CPU, Memory Peak, OOM, and PID Counters

```
cpu_stat = parse_key_value_file(cgroup_path / "cpu.stat")
usage_usec = cpu_stat.get("usage_usec")
user usec = cpu stat.get("user usec")
system usec = cpu stat.get("system usec")
if usage_usec is not None:
    metrics["cpu_total_sec"] = round(usage_usec / 1_000_000.0, 4)
metrics["memory_peak_bytes"] = read_int_file(cgroup_path / "memory.peak")
mem_events = parse_key_value_file(cgroup_path / "memory.events")
oom kill = mem events.get("oom kill", 0)
oom count = mem events.get("oom", 0)
oom_killed = (oom_kill > 0) or (oom_count > 0)
pids_events = parse_key_value_file(cgroup_path / "pids.events")
metrics["pids_max_events"] = pids_events.get("max", 0)
```

Source: sandbox_engine/monitoring/collector.py (collect_cgroup_v2_metrics).

Reading oom and oom_kill counters together guarantees accurate OOM classification across both kernel 5.15 and 6.x memory controller releases.

7.9 Async Worker Dispatch and State Mapping

Inside ExecutionManager._worker_loop(), synchronous supervisor execution shifts onto a background thread with asyncio.to_thread() so API calls remain responsive, then maps TerminationReason values to persistent job states.

Listing 7.9: Worker Thread Offloading and Terminal State Resolution

```
job.state = ExecutionState.RUNNING
await self._notify_update(job)
try:
    result: SupervisedExecutionResult = await asyncio.to_thread(
        self.runner.run_program,
        execution_id=job.id, code=job.source_code,
        policy=job.policy, cancel_check=lambda: job.id in self.cancelled_job_ids,
    )
    job.exit code = result.exit code
    job.metrics = result.metrics
    if result.termination_reason == TerminationReason.RESOURCE_LIMIT_EXCEEDED:
        job.state = ExecutionState.RESOURCE_LIMIT_EXCEEDED
except SandboxCapabilityError as cap_err:
    job.state = ExecutionState.FAILED
    job.termination_reason = TerminationReason.PREPARATION_ERROR
```

Source: sandbox_engine/manager/execution_manager.py (_worker_loop).

Offloading to asyncio.to_thread() keeps FastAPI HTTP endpoints responsive so users can click Cancel on a timed or runaway workload while the worker thread polls its 20 ms loop.

7.10 Sequential Policy Lab Execution and Persistence Bridge

Comparative benchmarking inside ExperimentService steps through each configured policy serially, polling execution completion every 0.5 seconds so CPU timing and OOM readings reflect isolated workload runs rather than parallel host contention.

Listing 7.10: Sequential Policy Lab Runner Execution Loop

```
for run in sorted(exp.runs, key=lambda r: r.run_order):
    run.status = "RUNNING"
    db.commit()
    exec rec = await self.execution service.submit execution(
        source_code=exp.source_code, policy_id=run.policy_id,
        custom_policy=None, language="python", db=db,
    )
    run.execution_id = exec_rec.id
    db.commit()
    while True:
        db.refresh(exec_rec)
        if exec_rec.state in (
            "COMPLETED", "FAILED", "TIMED_OUT",
            "CANCELLED", "RESOURCE LIMIT EXCEEDED",
        ):
            run.status = exec rec.state
            db.commit()
            break
        await asyncio.sleep(0.5)
```

Source: apps/api/app/services/experiment_service.py (run_experiment).

Calling db.refresh(exec_rec) inside the polling loop ensures SQLAlchemy reads updated state written by the separate SessionLocal transaction inside _handle_job_update().

7.11 Asynchronous Callback Synchronisation and Lifespan Recovery

Decoupling SQLAlchemy from sandbox_engine relies on two clean lifecycle hooks shown in Listing 7.11 below: _handle_job_update() in deps.py opens an independent session per state change, while FastAPI lifespan startup in main.py reconciles unfinished jobs after unexpected server termination.

Listing 7.11: Callback DB Synchronisation and Startup Orphan Reconciliation

```
async def _handle_job_update(job: ExecutionJob) -> None:
    db = SessionLocal()
    try:
        repo = ExecutionRepository(db)
        audit_repo = AuditRepository(db)
        repo.update_from_job(job)
        audit_repo.log_event(
            event_type=f"JOB_{job.state.value}", execution_id=job.id,
            details={"exit code": job.exit code},
        )
    finally:
        db.close()
# FastAPI lifespan startup reconciliation (main.py):
orphaned = db.query(ExecutionRecord).filter(
    ExecutionRecord.state.in ([“QUEUED”, “PREPARING”, “RUNNING”])
).all()
for rec in orphaned:
    rec.state = "CANCELLED"
    rec.termination_reason = "INTERRUPTED"
```

Source: apps/api/app/api/deps.py and apps/api/app/main.py (lifespan recovery).

Reconciling orphaned jobs on startup ensures the Executions audit log and Overview counters never hang on stale RUNNING indicators after a laptop reboot or uvicorn reload.

7.12 Ubuntu Deployment Provisioning and Verification Procedure

Deploying Sandboxed on Ubuntu 22.04 or 24.04 LTS uses setup_ubuntu.sh (run once under sudo to install bubblewrap, libseccomp, Python venv, and Node.js 20 and delegate /sys/fs/cgroup/sandboxed.slice) followed by unprivileged verification and startup commands shown in Listing 7.12.

Listing 7.12: Reproducible Ubuntu Provisioning and Sandbox Verification

```
SANDBOX SLICE="/sys/fs/cgroup/sandboxed.slice"
mkdir -p "$SANDBOX SLICE"
echo "+cpu +memory +pids" > "$SANDBOX_SLICE/cgroup.subtree_control" || true
chown -R "$REAL_USER:$REAL_USER" "$SANDBOX_SLICE"
chmod -R 775 "$SANDBOX_SLICE"
# Unprivileged operator verification and launch:
./scripts/verify linux sandbox.sh
./scripts/run_dev.sh
```

Source: scripts/setup_ubuntu.sh and scripts/ (verify_linux_sandbox.sh, run_dev.sh).

Once setup_ubuntu.sh exits cleanly, verify_linux_sandbox.sh prints the live detect_capabilities() report, runs a direct bubblewrap container smoke test binding only read-only /usr, /lib, /lib64, and /bin, and executes the complete pytest suite including all 10 Linux kernel integration tests before launching Uvicorn and Vite.

8. System Verification, Features, and Interface Walkthrough

8.1 Overview Dashboard and Capabilities Status Banner

Figures 8.1 through 8.7 capture the running React interface (127.0.0.1:5173) connected to the FastAPI backend (127.0.0.1:8000) on Windows 11. On the Overview view, KPI tiles summarize total jobs, completions, timeouts, and resource terminations alongside a live diagnostic strip explaining why kernel isolation stays locked until Ubuntu deployment.

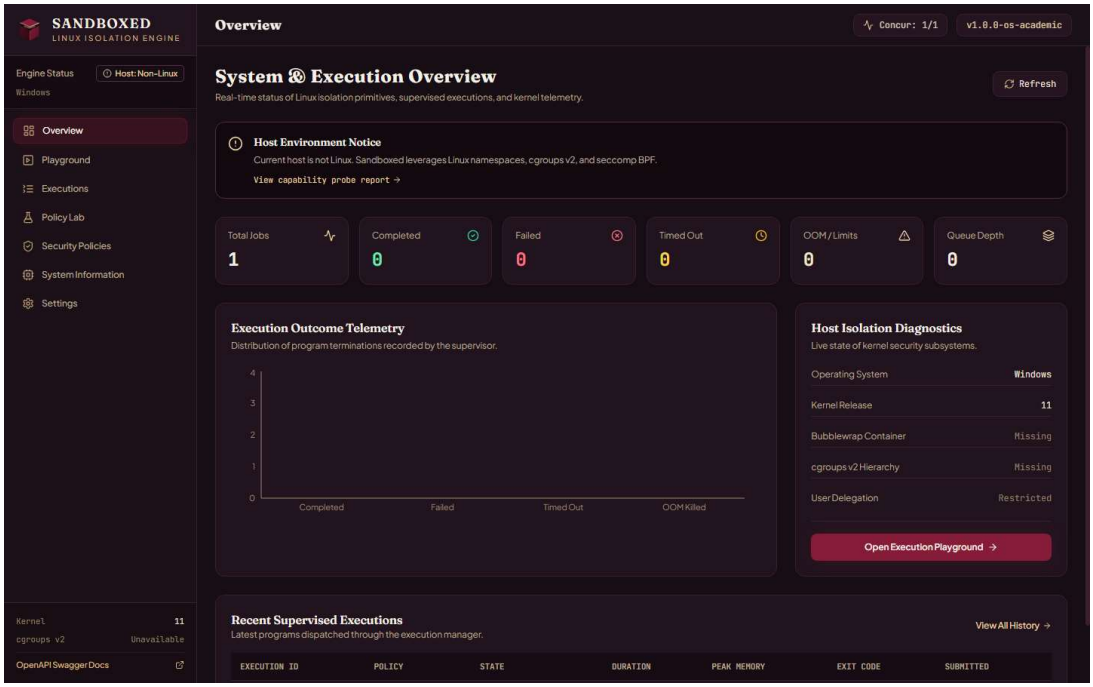


Figure 8.1: System and Execution Overview Dashboard

Description: Main Overview route showing non-Linux host status banner, aggregate execution summary cards, outcome telemetry chart, and capability readiness breakdown on Windows 11.

Inspecting Figure 8.1 confirms that live dashboard telemetry polls SQLite via GET /api/dashboard/summary without stale client caches. Even when execution attempts fail readiness checks on Windows, the counter cards and bar plot categorize outcomes cleanly so instructors see queue depth and active workers at a glance.

8.2 Interactive Execution Playground and Policy Catalogue

The Playground workspace pairs a Python CodeMirror editor with preset dropdown selection for all eight built-in workloads, .py file upload, and live policy targeting. Beneath and alongside the editor, inspectors display stdout, stderr, and execution metrics once runs conclude. The companion Policies view lists all seeded and custom profiles, clearly marking standard and strict as core immutable entries.

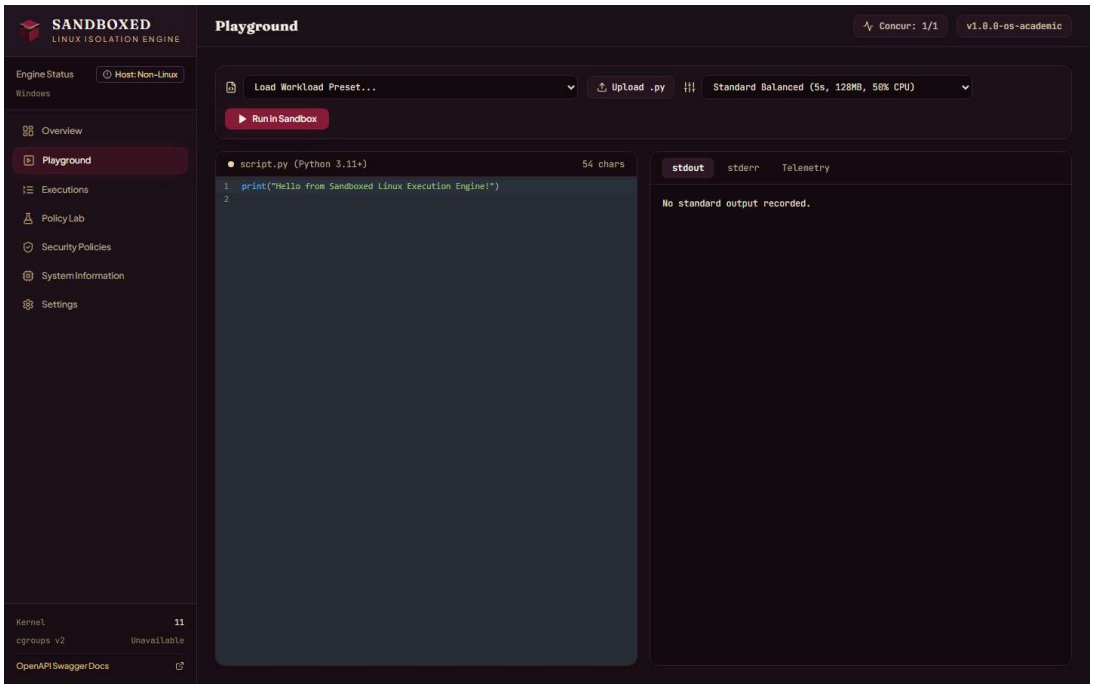


Figure 8.2: Interactive Execution Playground

Description: Playground page showing the CodeMirror Python editor, workload preset selector, file upload trigger, execution policy dropdown, and stdout/stderr/metrics inspection panels.

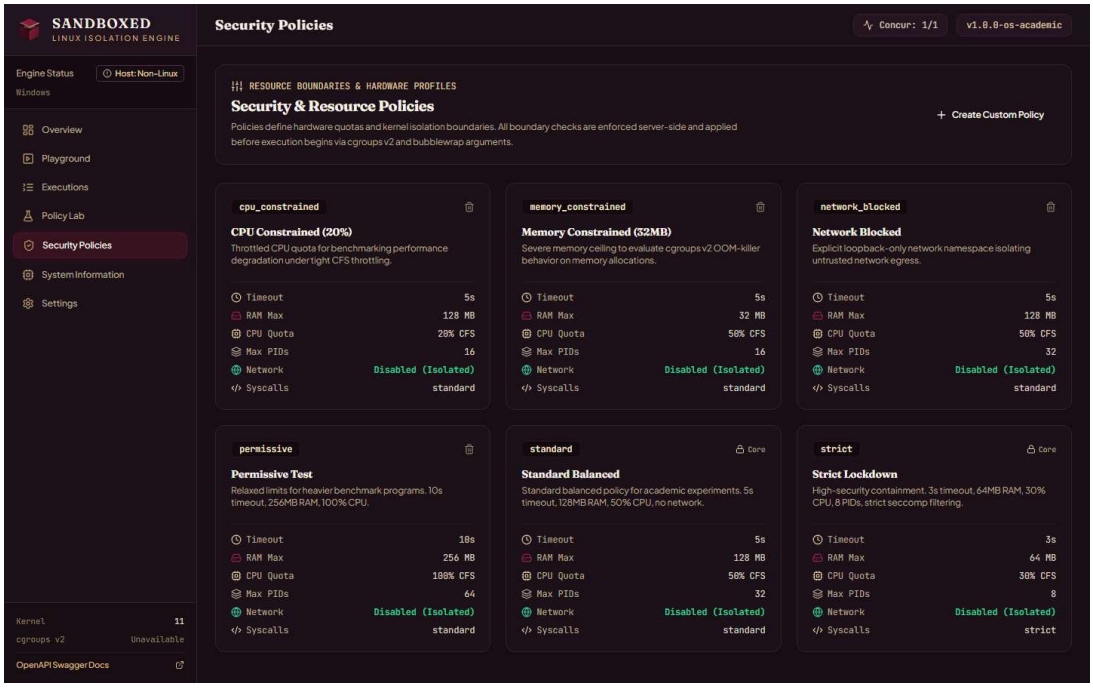


Figure 8.3: Security and Resource Policy Profiles

Description: Policies catalogue presenting cpu_constrained, memory_constrained, network_blocked, permissive, standard, and strict profiles with core protection indicators and resource limit badges.

Switching presets in Playground loads validated template scripts straight from GET /api/system/workloads rather than duplicating source code inside React bundles. Meanwhile, the Policies page lets students edit or remove custom profiles while rendering disabled delete controls on core standard and strict presets.

8.3 Execution Audit Log and Stream Inspector

Every job persisted inside SQLite remains searchable on the Executions route across filter tabs (ALL, COMPLETED, FAILED, TIMED_OUT, RESOURCE_LIMIT_EXCEEDED, CANCELLED, RUNNING). Clicking an entry loads its UUID, source SHA-256 hash, termination code, captured streams, cgroups telemetry, and immutable JSON snapshot.

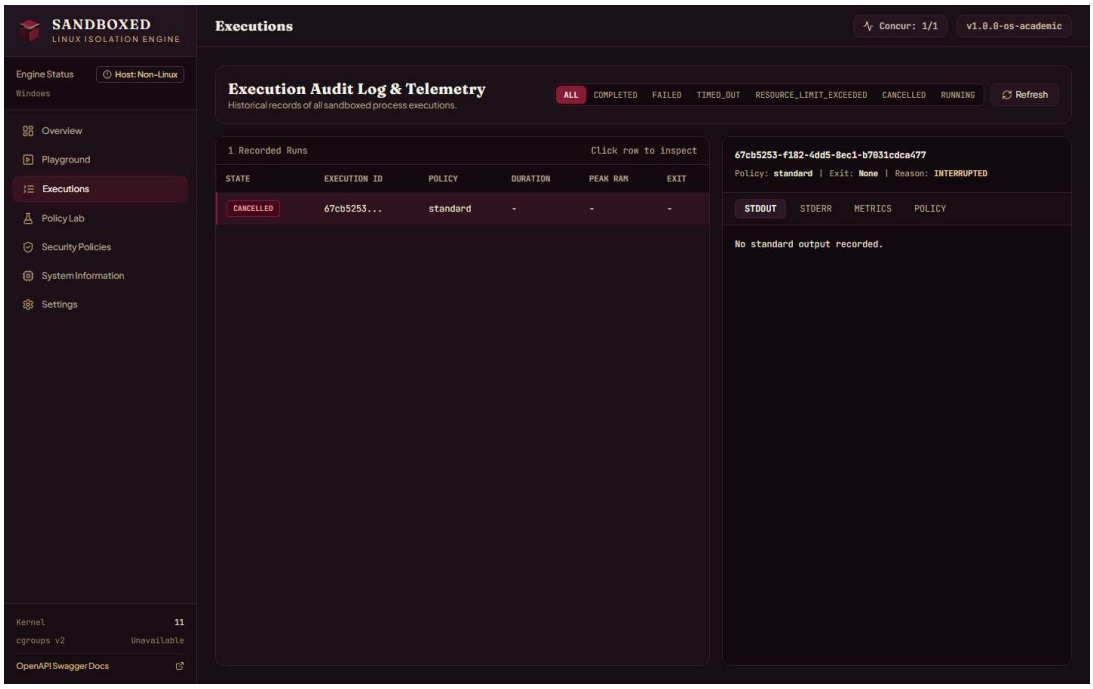


Figure 8.4: Execution Audit Log and Stream Inspector

Description: Executions view listing recorded jobs alongside STDOUT, STDERR, METRICS, and POLICY inspection tabs, including startup orphan reconciliation entries marked INTERRUPTED.

Figure 8.4 also demonstrates startup orphan recovery in practice: prior interrupted sessions render cleanly under the CANCELLED filter tab with termination_reason recorded as INTERRUPTED alongside their frozen policy.

8.4 Policy Lab Comparative Experiments and Host Diagnostics

The Policy Lab screen lets an operator choose a stress script, check between two and six policy profiles, and trigger sequential execution through /api/experiments/{id}/run. Completed runs update the comparative table and Recharts bar plot automatically.

Meanwhile, System Info exposes full detect_capabilities() telemetry and FastAPI serves OpenAPI 3.1 Swagger docs at /docs.

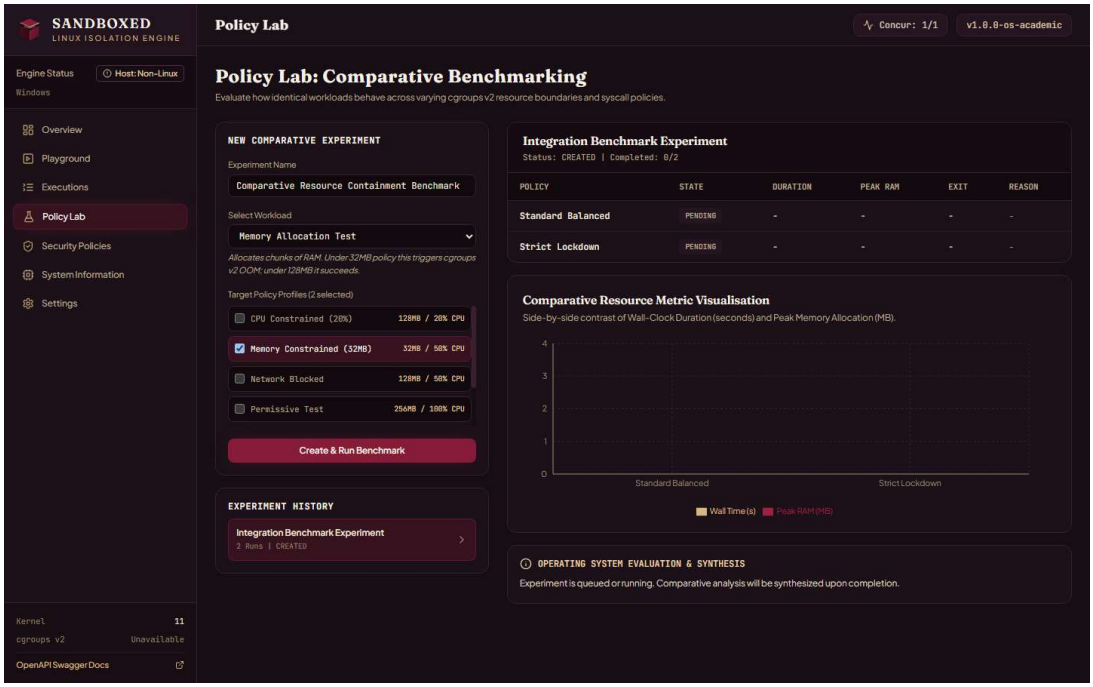


Figure 8.5: Policy Lab Comparative Benchmarking Workspace

Description: Policy Lab interface showing experiment builder controls, multi-policy selection, side-by-side metrics table, Recharts bar axis, and synthesized evaluation panel.

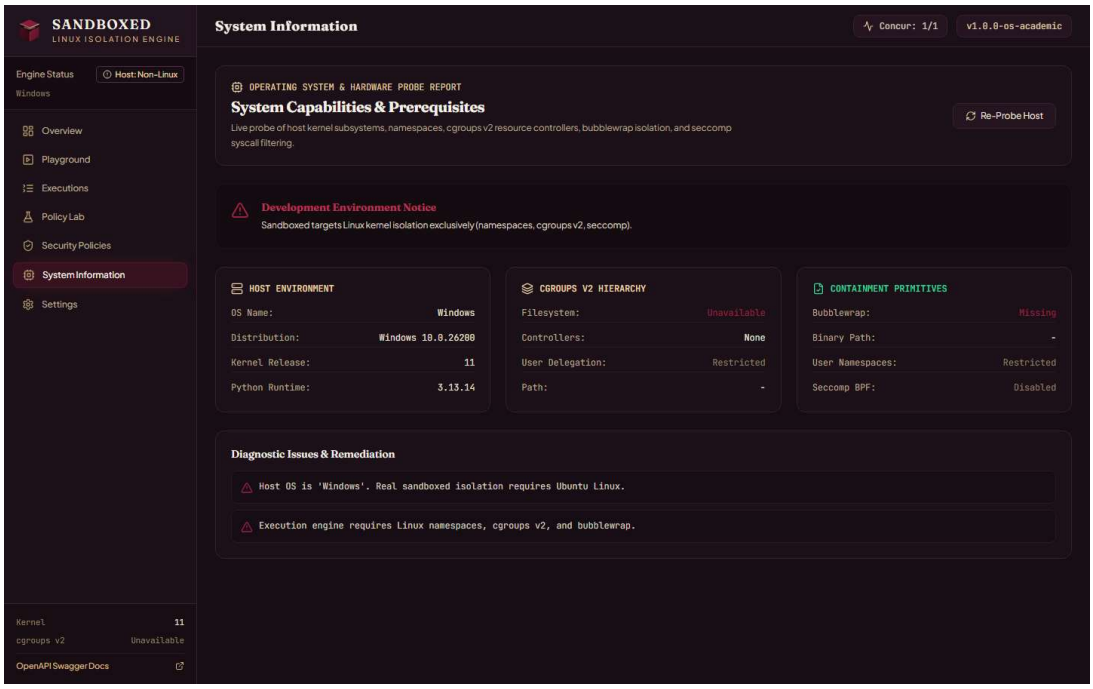


Figure 8.6: Live Host Capability Probe and Diagnostics

Description: System Information view exposing live detect_capabilities() telemetry on Windows 10.0.26200 (Python 3.13.14), flagging missing bubblewrap and cgroups v2 readiness.

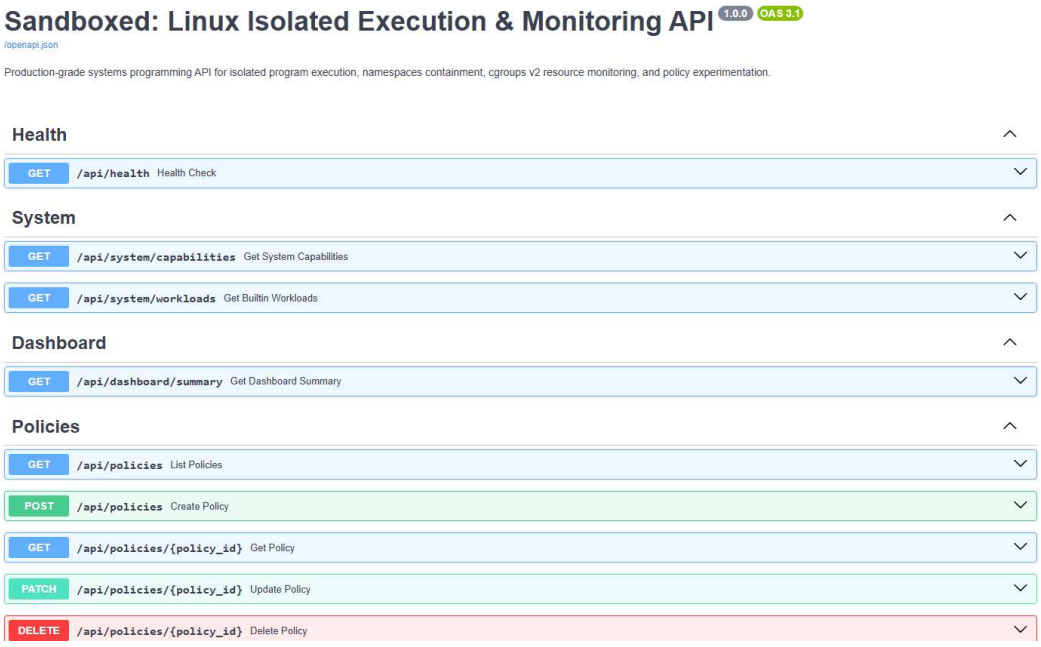


Figure 8.7: FastAPI OpenAPI 3.1 Interactive Documentation

Description: Swagger UI at http://127.0.0.1:8000/docs exposing interactive endpoint schemas for Health, System, Dashboard, Policies, Executions, and Policy Lab Experiments.

System Info (Figure 8.6) surfaces exact host diagnostics returned by GET /api/system/capabilities, while Swagger UI (Figure 8.7) allows automated API integration scripts and instructors to invoke endpoints directly.

8.5 Automated Test Verification Summary (Windows Phase)

Running .venv\Scripts\python.exe -m pytest -v across apps/api/tests and sandbox_engine/tests executed all 53 collected tests in 8.06 seconds on Windows 11. Forty-three tests passed with zero failures: 18 API, policy validation, and SQLite repository checks (Table 8.1) plus 25 core sandbox engine unit tests (Table 8.2). The remaining 10 Linux integration cases in test_linux_integration_suite.py reported SKIPPED via sys.platform.

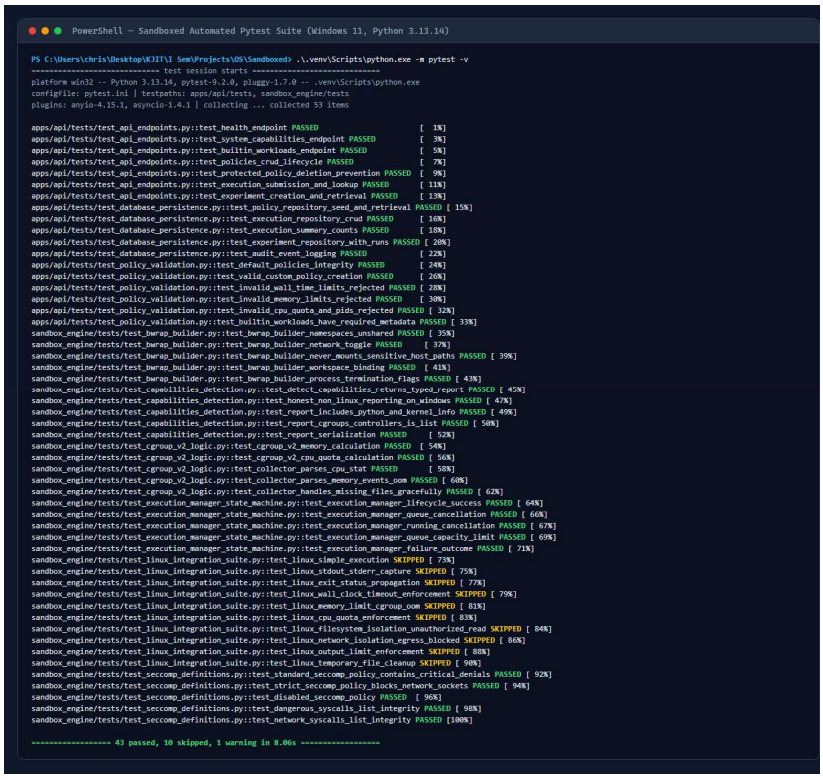


Figure 8.8: Verified Pytest Execution Output (43 Passed, 10 Linux Checks Skipped)

Description: Terminal capture of pytest verbose execution reporting 43 passed unit and API verification tests and 10 platform-guarded Linux integration skips on Windows win32.

Table 8.1: Backend API, Persistence, and Policy Test Inventory (18 Passed on Windows)

Test File & Outcome	Exact Pytest Function Names	Verified Assertion Scope
test_api_endpoints.py (7 PASSED)	test_health_endpoint, test_system_capabilities_endpoint, test_builtin_workloads_endpoint, test_policies_crud_lifecycle	HTTP status codes, OpenAPI health JSON, 8 builtin workloads, policy POST/GET/PATCH/DELETE round-trips

Test File & Outcome	Exact Pytest Function Names	Verified Assertion Scope
test_api_endpoints.py (cont.)	test_protected_policy_deletion_prevention, test_execution_submission_and_lookup, test_experiment_creation_and_retrieval	HTTP 400 on deleting standard / strict, async execution 201 submit + GET lookup, experiment creation
test_database_persistence.py (5 PASSED)	test_policy_repository_seed_and_retrieval, test_execution_repository_crud, test_execution_summary_counts, test_experiment_repository_with_runs, test_audit_event_logging	SQLite table creation, policy seeding, summary counts by state, CASCADE run linkage, audit log timestamps
test_policy_validation.py (6 PASSED)	test_default_policies_integrity, test_valid_custom_policy_creation, test_invalid_wall_time_limits_rejected, test_invalid_memory_limits_rejected, test_invalid_cpu_quota_and_pids_rejected, test_builtin_workloads_have_required_metadata	Pydantic boundary checks (0.1..60s, 16..2048 MiB, 10..100% CPU, 1..128 PIDs) and workload schemas

Table 8.2: Core Sandbox Engine Unit Test Inventory (25 Passed on Windows)

Test File & Outcome	Exact Pytest Function Names	Verified Assertion Scope
test_bwrap_builder.py (5 PASSED)	test_bwrap_builder_namespaces_unshared, test_bwrap_builder_network_toggle, test_bwrap_builder_never_mounts_sensitive_host_paths, test_bwrap_builder_workspace_binding, test_bwrap_builder_process_termination_flags	--unshare-* flags, --unshare-net vs --share-net, absence of /home or /root mounts, /workspace chdir, --die-with-parent
test_capabilities_detection.py (5 PASSED)	test_detect_capabilities_returns_typed_report, test_honest_non_linux_reporting_on_windows, test_report_includes_python_and_kernel_info, test_report_cgroups_controllers_is_list, test_report_serialization	Typed EngineCapabilityReport fields, honest ready=False on win32, kernel / Python versions, JSON output
test_cgroup_v2_logic.py (5 PASSED)	test_cgroup_v2_memory_calculation, test_cgroup_v2_cpu_quota_calculation, test_collector_parses_cpu_stat, test_collector_parses_memory_events_oom, test_collector_handles_missing_files_gracefully	Byte & CFS microsecond math, cpu.stat division by 1e6, oom_kill attribution, missing pseudo-file tolerance
test_execution_manager_state_machine.py (5 PASSED)	test_execution_manager_lifecycle_success, test_execution_manager_queue_cancellation, test_execution_manager_running_cancellation, test_execution_manager_queue_capacity_limit, test_execution_manager_failure_outcome	QUEUED -> COMPLETED flow, queued & active cancellation, capacity 50 rejection, non-zero exit FAILED state
test_seccomp_definitions.py (5 PASSED)	test_standard_seccomp_policy_contains_critical_denials, test_strict_seccomp_policy_blocks_network_sockets, test_disabled_seccomp_policy, test_dangerous_syscalls_list_integrity, test_network_syscalls_list_integrity	18 dangerous syscalls in STANDARD, 28 merged calls in STRICT, empty denylist in DISABLED

8.6 Ubuntu Linux Kernel Benchmark Protocol and Troubleshooting Guide

No synthetic Linux timings are fabricated in this report. On an Ubuntu 22.04 or 24.04 LTS host or VMware VM configured via scripts/setup_ubuntu.sh and checked via scripts/verify_linux_sandbox.sh, the 10 skipped tests in test_linux_integration_suite.py unlock and validate live containment. Table 8.3 catalogs all 10 integration tests alongside their paired benchmark policy conditions.

Each Policy Lab stress condition targets a measurable physical threshold in the Linux kernel. Under the 32 MiB memory_constrained profile, setting memory.swap.max=0 forces the cgroup out-of-memory killer to terminate memory_allocation roughly around the sixth or seventh 5 MiB chunk, writing oom_kill=1 in memory.events while leaving the host API daemon untouched. Under the 20 percent cpu_constrained profile, cpu.max writes 20000 100000, restricting a CPU-bound prime search to 20 ms out of every 100 ms scheduling period so ExperimentService directly quantifies CFS bandwidth throttling in seconds.

Table 8.3: Linux Kernel Integration Tests & Ubuntu Benchmark Protocol (10 Pending Linux Run)

Pytest Integration Function	Stress Condition	Configured Limit	Expected Kernel Enforcement
test_linux_simple_execution, _stdout_stderr_capture, _exit_status_propagation	Normal script & error streams	standard (5s, 128 MiB)	Accurate exit code and separate stream capture
test_linux_wall_clock_timeout_enforcement	deadline_timeout (19s sleep)	wall_time_limit_sec = 1..3s	TIMED_OUT; supervisor kills -pgid cleanly
test_linux_memory_limit_cgroup_oom	memory_allocation (50 MiB)	memory_constrained (32 MiB)	RESOURCE_LIMIT_EXCEEDED (oom_kill >= 1)
test_linux_cpu_quota_enforcement	cpu_intensive (25,000 primes)	cpu_constrained (20%) vs 100%	Measured CFS throttling variance in Policy Lab
test_linux_filesystem_isolation_unauthorized_read	file_escape_attempt probes	isolated_ro_root / strict_ro	Host files absent; PermissionError / FileNotFoundError
test_linux_network_isolation_egress_blocked	network_attempt (8.8.8.8:53)	network_blocked (--unshare-net)	ENETUNREACH / socket timeout inside isolated net
test_linux_output_limit_enforcement, _temporary_file_cleanup	output_limit_test + teardown check	max_output_bytes + rmdir	truncated_output=True; zero leftover cgroup/tmp

When provisioning a fresh Ubuntu host, operators occasionally hit host-level kernel defaults such as legacy cgroup v1 mounts, restricted unprivileged user namespaces on Ubuntu 24.04 AppArmor, or unowned cgroup delegation slices. Table 8.4 maps each pre-flight warning reported by detect_capabilities() to its fix.

Table 8.4: Ubuntu Host Verification Troubleshooting Matrix

Diagnostic Symptom / Issue	Root Host Cause	Verified Remediation Step
cgroups v2 filesystem not found at /sys/fs/cgroup	Host booted into legacy hybrid cgroups v1	Set systemd.unified_cgroup_hierarchy=1 in GRUB and reboot
Unprivileged user namespaces disabled	sysctl unprivileged_userns_clone=0 or AppArmor rule	Run setup_ubuntu.sh or allow bwrap AppArmor userns profile
No write permissions to delegate cgroups	sandboxed.slice missing or owned by root:root	Run sudo ./scripts/setup_ubuntu.sh to chown slice to user
Controller 'memory' or 'pids' missing	Subtree delegation omitted in root cgroup.subtree_control	Echo '+cpu +memory +pids' into /sys/fs/cgroup/cgroup.subtree_control

9. Future Enhancements

9.1 Hardware-Virtualised Micro-VM Runners (Firecracker and gVisor)

Unprivileged namespaces and cgroups v2 provide strong boundaries against ordinary malicious user payloads, yet every contained task still shares system-call entry points inside one host Linux kernel. Adding a backend adapter behind ExecutionManager for KVM-backed Firecracker micro-VMs or gVisor user-space kernels would give each job its own guest kernel while preserving identical Policy Lab schemas and SQLite telemetry tables.

Keeping runner interfaces abstract behind LinuxSandboxRunner makes this upgrade clean: fast academic runs can select lightweight bwrap isolation with millisecond startup, while high-risk binary analysis jobs route to a micro-VM backend exposing /dev/kvm virtualisation.

9.2 Runtime Seccomp BPF Compilation and Landlock Integration

BwrapCommandBuilder already exposes an optional seccomp_bpf_fd parameter pointing to --seccomp, and seccomp_filter.py defines both Standard (18 calls) and Strict (28 calls) denylists. Compiling those declarative definitions into raw BPF bytecode via libseccomp at execution time, then passing the read-only memfd descriptor into LinuxSandboxRunner, completes kernel system-call filtering. Pairing seccomp with Linux Landlock ABI paths would add fine-grained kernel pathname rules beneath /workspace as a secondary defence layer.

A direct advantage of seccomp-bpf filters compiled per execution is auditability: denied syscalls can return deterministic EPERM errors or emit SECCOMP_RET_LOG kernel trace lines for security students studying syscall graphs.

9.3 Multi-Language Compiler Toolchains

ExecutionRecord and ExecutionService already persist a language column set to python by default. Introducing dedicated two-phase compile-and-run profiles inside BwrapCommandBuilder for C/C++ (gcc/clang with strict compile-time memory caps), Rust (rustc), and Node.js would let instructors evaluate systems programming assignments alongside Python coursework inside the same browser console.

Isolating the compiler stage inside its own transient cgroup prevents classic compiler-bomb template abuse, such as recursive C++ constexpr expansion from exhausting host memory during grading.

9.4 Real-Time WebSocket Stream Tail and Multi-Node Broker Scheduling

Current clients query GET /api/executions/{execution_id} repeatedly to inspect status transitions and output streams after completion. Exposing a FastAPI WebSocket endpoint tied to non-blocking supervisor pipe reads would stream stdout chunks live into the Playground terminal during long loops. For department-wide grading queues, swapping the single-process asyncio.Queue for Redis or RabbitMQ workers lets multiple Linux runner nodes scale horizontally.

10. Conclusion

10.1 Engineering Synthesis and Operating Systems Insights

Sandboxed illustrates how core Linux kernel facilities interlock when engineered into a unified platform. Rather than examining namespaces, cgroups v2, seccomp denylists, and POSIX signals as isolated textbook definitions, the project connects them through validated Pydantic policy records, explicit bwrap command vectors, non-blocking descriptor supervision, and SQLite persistence.

Implementing the stack revealed critical systems invariants that dictate reliability in practice. Kernel counters inside cpu.stat, memory.peak, memory.events, and pids.events vanish the instant a slice directory is unlinked, so ProcessSupervisor must harvest telemetry before cleanup_cgroup() deletes the slice. Standard output pipes must use O_NONBLOCK reads bounded below max_output_bytes so runaway print loops cannot stall the timeout watchdog. Orphaned jobs left inside SQLite during abrupt daemon exits require startup reconciliation, and host probing must never disguise missing Linux primitives behind fake execution success.

A further lesson involves race closure during process creation. Assigning a child PID into cgroup.procs both inside POSIX preexec_fn right after fork() and again from the parent after Popen eliminates early allocation bursts before accounting attaches, while os.killpg(-pgid) plus cgroup.kill extinguishes orphan subshells.

10.2 Known Constraints and Architectural Tradeoffs

Two explicit constraints define the scope of this release. First, physical enforcement of unprivileged namespaces and cgroups v2 limits requires deployment on Ubuntu 22.04 or 24.04 LTS; on the Windows 11 development workstation, all 43 platform-independent tests pass while the 10 Linux integration checks stay explicitly skipped and unprivileged capability checks decline execution cleanly. Second, seccomp policy profiles are currently verified as declarative syscall denylist models inside seccomp_filter.py while BwrapCommandBuilder exposes the optional --seccomp FD parameter for runtime BPF attachment.

Choosing single-worker sequential dispatch (MAX_CONCURRENCY=1) trades horizontal throughput for measurement precision: every Policy Lab comparison measures CFS CPU and peak memory free from noisy parallel contention.

10.3 Final Summary of Outcomes

Within its academic and laboratory charter, Sandboxed delivers transparent kernel containment without root daemons, strict server-validated resource bounds, automated SQLite audit trails with frozen policy snapshots, and interactive side-by-side policy comparison via the Policy Lab browser interface.

References

[1] Linux Programmer's Manual, "namespaces(7): Overview of Linux namespaces," Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man7/namespaces.7.html

[2] Linux Programmer's Manual, "cgroups(7): Linux control groups," Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man7/cgroups.7.html

[3] T. Heo and Linux Kernel Documentation Contributors, "Control Group v2 (cgroups-v2)," The Linux Kernel Archives. [Online]. Available: https://docs.kernel.org/admin-guide/cgroup-v2.html

[4] Linux Kernel Documentation, "CFS Bandwidth Control," The Linux Kernel Archives. [Online]. Available: https://docs.kernel.org/scheduler/sched-bwc.html

[5] Linux Programmer's Manual, "seccomp(2): Operate on Secure Computing state of the process," Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man2/seccomp.2.html

[6] Linux Programmer's Manual, "setsid(2) and killpg(3): Session and Process Group Signal Control," Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man2/setsid.2.html

[7] Project Atomic / Containers Contributors, "Bubblewrap (bwrap): Unprivileged sandboxing tool," GitHub Repository. [Online]. Available: https://github.com/containers/bubblewrap

[8] S. Ramirez, "FastAPI Documentation: Asynchronous REST APIs & Pydantic v2 Integration," Tiangolo. [Online]. Available: https://fastapi.tiangolo.com/

[9] Pydantic Contributors, "Pydantic V2 Documentation: Data Validation Using Python Type Annotations," Pydantic Services Inc. [Online]. Available: https://docs.pydantic.dev/latest/

[10] M. Bayer, "SQLAlchemy 2.0 Documentation: ORM Mapping & SQLite Persistence," SQLAlchemy Project. [Online]. Available: https://docs.sqlalchemy.org/

[11] D. R. Hipp et al., "SQLite Documentation: Appropriate Uses and Transactional Storage," SQLite Consortium. [Online]. Available: https://www.sqlite.org/docs.html

[12] Meta Platforms & Vite Contributors, "React 18 and Vite Frontend Build Tooling," React Documentation. [Online]. Available: https://react.dev/

[13] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, No Starch Press, 2010.

[14] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/